

ACTIVITIES PLAN

Global Digital Bank

Iterative Project Implementation Plan

A structured, progressive series of **20 activities** guiding students from core Java fundamentals to full-stack enterprise application development.

Bank Account Management System – Activities Plan

Activity #	Title	Description	Key Tech Concepts (Tags)	Implementation Elements
1	Basic Account Class	Create a simple Account class that stores account info and supports deposit/withdraw	OOP, Class, Object, Encapsulation	Account, fields, constructor, deposit(), withdraw(), getters, setters
2	Testing Account Class	Write a test driver to create accounts and verify deposit/withdraw flows	Testing, Main Method, Console I/O	TestAccount, main(), object creation, method calls, formatted output
3	Enhanced Account Class	Add validation (age ≥ 18, min balance), PIN protection, and status management	Validation, State Management, Business Rules	AccountEnhanced, PIN field, setPin(), verifyPin(), closeAccount(), reopenAccount()
4	Testing Enhanced Account	Test validation, PIN verification, status transitions, and error scenarios	Testing, Edge Cases, Test Coverage	TestAccountEnhanced, scenario-based test cases, expected output validation
5	Exceptions in Account Class	Replace boolean returns with meaningful exceptions for error conditions	Exceptions, Custom Exceptions, Error Handling	AccountException, InvalidAmountException, InsufficientBalanceException, MinimumBalanceViolationException, InactiveAccountException, InvalidPinException
6	Testing Exceptions	Write test code with try-catch to verify all exception scenarios	Try-Catch, Exception Handling, Error Reporting	TestAccountExceptions, try-catch blocks, exception message validation
7	Account Subclasses	Create SavingsAccount and CurrentAccount extending base Account	Inheritance, Extends, Super, Method Overriding	SavingsAccount, CurrentAccount, super(), overridden methods, specialized fields
8	Testing Subclasses	Test specialized behavior (interest, overdraft) and polymorphism	Polymorphism, Dynamic Dispatch, Type-Specific Testing	TestAccountSubclasses, polymorphic references, type-specific method tests
9	Abstract BankAccount	Refactor to abstract BankAccount with abstract methods for subclass contracts	Abstract Class, Abstract Methods, Template Method, Contract	BankAccount, getMinimumBalance(), getAccountType(), getInterestRate(), canWithdraw(), applyMonthlyInterest()
10	Testing Abstract Class	Test abstraction, polymorphism, and type-specific rules across all account types	Polymorphism, Abstract Dispatch, Type-Specific Rules	TestAbstractAccount, accounts array, dynamic method invocation
11	IAccount Interface & Factory	Introduce IAccount interface, AbstractAccount, and AccountFactory for creation	Interface, Factory Pattern, Loose Coupling, Abstraction	IAccount, AbstractAccount, AccountFactory, SavingsAccount, CurrentAccount, FixedDepositAccount, SalaryAccount
12	Testing Interface & Factory	Test interface-based operations and factory-based account creation	Interface Polymorphism, Factory Usage, Runtime Selection	TestInterfaceFactory, factory-driven creation, interface references
13.1	Rules Engine – If-Else	Build AccountRulesEngine using if-else for tenure-based rules (min balance, interest)	Singleton, Conditional Logic, Tenure Buckets, Real Bank Data	AccountRulesEngine, getMinimumBalance(), getInterestRate(), getFeatureName(), getTenureBucketName()
13.2	Rules Engine – Map-Based	Refactor rules engine to use nested Map for cleaner lookup	Map Data Structure, Inner Class, O(1) Lookup, Refactoring	AccountRulesEngine.Rule, Map<String, Map<Integer, Rule>>, loadRules()
14	Rules Engine – Properties	Move rules to .properties files, load dynamically, add hot reload	Properties Files, Externalized Config, Resource Loading, Hot Reload	AccountRulesPropertiesLoader, savings.properties, current.properties, fixeddeposit.properties, salary.properties, reloadRules()
15	Transfer with Daily Limits	Implement TransferService with privilege-based daily transfer limits	Service Layer, Atomic Operations, Daily Limits, Privilege Rules	TransferService, dailyTransferTotal, lastTransferDate, getDailyTransferLimit(), canTransfer(), new properties daily.transfer.limit.*
16	Transaction Model	Introduce Transaction class and TransactionType enum; overload methods to return transactions	Domain Modeling, Enums, Method Overloading, Coexistence	Transaction, TransactionType (DEPOSIT, WITHDRAW, TRANSFER), new overloaded methods returning Transaction
17	Command Pattern + File Logging	Wrap Deposit/Withdraw/Transfer as Commands; serialize to file	Command Pattern, Java Serialization, File I/O, Encapsulation	TransactionCommand, DepositCommand, WithdrawCommand, TransferCommand, TransactionLog (ObjectOutputStream, ObjectInputStream)
18	Bridge Pattern – File + DB	Abstract logging with LogDestination interface; support File, Memory, and DB	Bridge Pattern, Abstraction vs Implementation, Runtime Switching, Simulated DB	TransactionLogger, LogDestination, FileLogDestination, DatabaseLogDestination, MemoryLogDestination, SimulatedDatabase
19	AccountService	Service layer with open/close/deposit/withdraw/transfer that calls domain and logs	Service Layer, Orchestration, Separation of Concerns, Single Entry Point	AccountService, account registry, openAccount(), closeAccount(), deposit(), withdraw(), transfer(), integration with TransactionLogger
20	AccountUI	Menu-driven console UI collecting input, delegating to AccountService, displaying results	UI Layer, Menu Design, User Input, Delegation, End-to-End Flow	AccountUI, start(), displayMainMenu(), handlers for each operation, Scanner-based input, Main launches UI

Basic Account Class

Create a simple Account class that stores account info and supports deposit/withdraw operations.

Key Concepts

- OOP
- Class
- Object
- Encapsulation

Implementation Elements

- Account
- fields (accountNumber, holderName, balance)
- constructor
- deposit()
- withdraw()
- getters
- setters

Code Snapshot

```
public class Account {  
    private String accountNumber;  
    private String holderName;  
    private double balance;  
  
    public Account(String accNo, String name, double  
balance) {  
        this.accountNumber = accNo;  
        this.holderName = name;  
        this.balance = balance;  
    }  
  
    public void deposit(double amount) {  
        balance += amount;  
    }  
  
    public void withdraw(double amount) {  
        balance -= amount;  
    }  
}
```

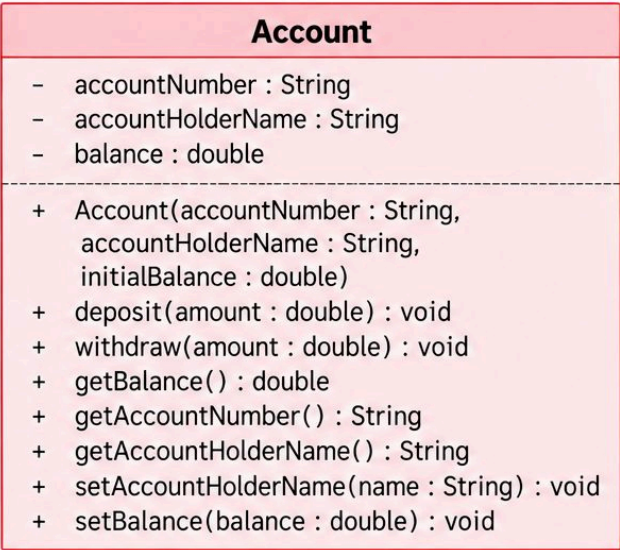
Account

- **Fields:**
 - accountNumber: String
 - holderName: String
 - balance: double
- **Methods:**
 - deposit()
 - withdraw()
 - getters/setters

Activity 1 Class Diagram

Activity 1 – Basic Account Class (Class Diagram)

Objective: Create a simple Account class that stores account information and supports deposit and withdraw operations.



Key Points

- Simple Account class with essential account information.
- Supports deposit and withdraw operations.
- Encapsulates fields using private access.
- Provides constructor, getters and setters.
- No validation, PIN, status or account types in this activity (these come in later activities).

Implementation Elements (from plan)

- Account class
- Fields: accountNumber, accountHolderName, balance
- Constructor
- deposit()
- withdraw()
- Getters and setters

Design Concepts Demonstrated

- OOP (Class and Object)
- Encapsulation (private fields, public methods)
- Constructor
- Instance methods
- Basic domain modeling (Account)
- Foundation for future enhancements (validation, exceptions, inheritance, etc.)

Example Usage (Test Driver in next activity)

```
Account acc = new Account("ACC1001", "John Doe", 1000.0);
acc.deposit(500.0);
acc.withdraw(200.0);
System.out.println("Balance: " + acc.getBalance());
```

Testing Account Class

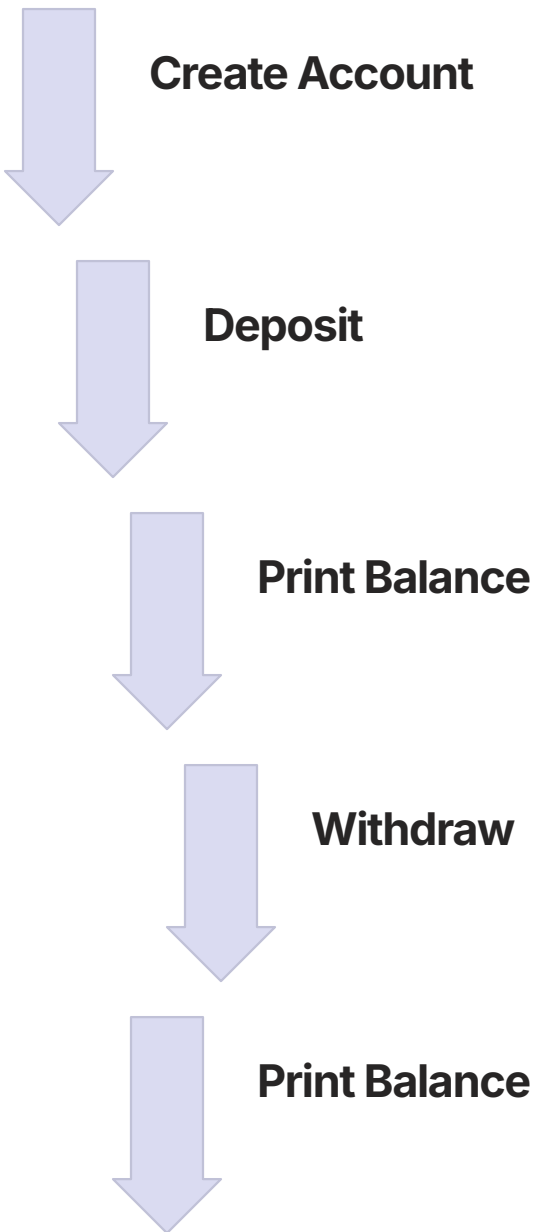
Write a test driver to create accounts and verify deposit/withdraw flows using console output.

Key Concepts

- Testing
- Main Method
- Console I/O

Implementation Elements

- TestAccount
- main()
- object creation
- method calls
- formatted output



Code Snapshot

```
public class TestAccount {
    public static void main(String[] args) {
        Account acc = new Account("A001", "Alice", 1000.0);
        acc.deposit(500.0);
        System.out.println("Balance: " + acc.getBalance());
        acc.withdraw(200.0);
        System.out.println("Balance: " + acc.getBalance());
    }
}
```

Enhanced Account Class

Add validation (age ≥ 18, min balance), PIN protection, and account status management.

Key Concepts

- Validation
- State Management
- Business Rules

Implementation Elements

- AccountEnhanced
- PIN field
- setPin()
- verifyPin()
- closeAccount()
- reopenAccount()
- AccountStatus enum

Code Snapshot

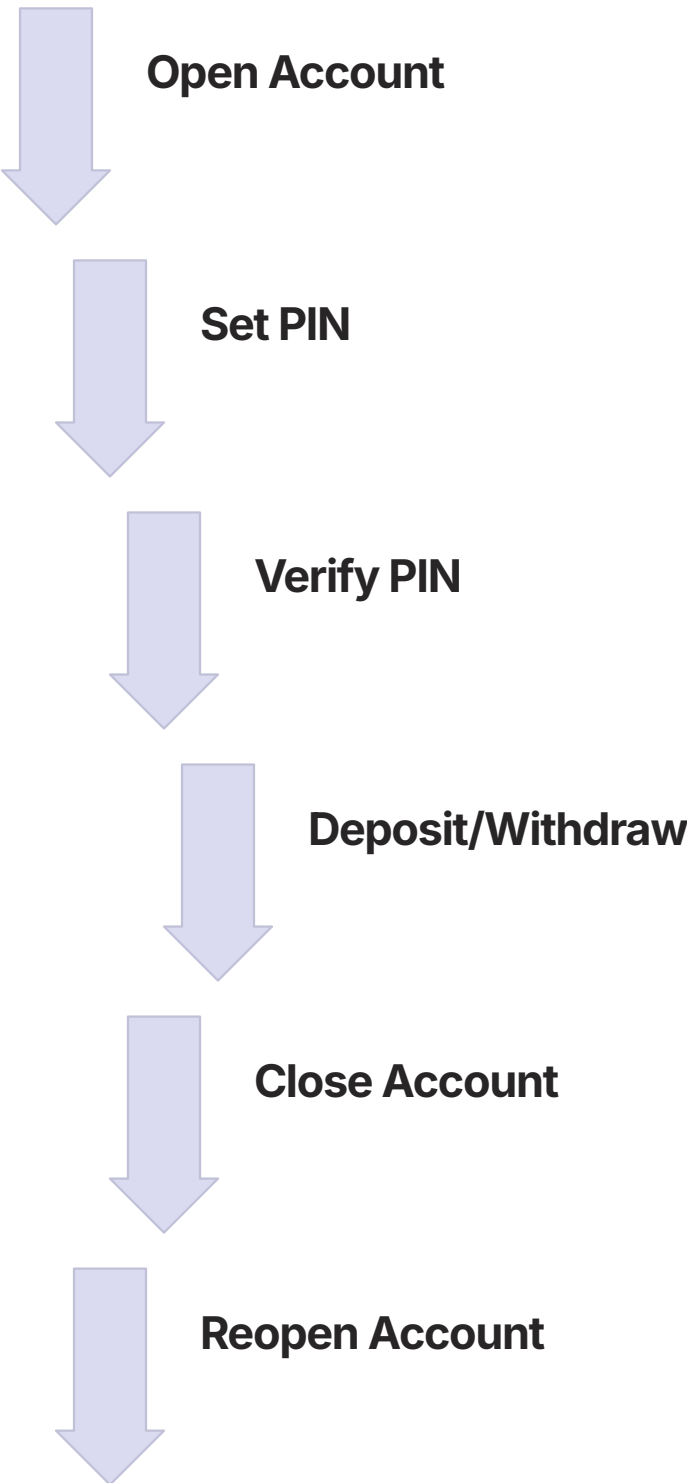
```
public class AccountEnhanced {
    private String pin;
    private AccountStatus status;
    private double minBalance = 500.0;

    public void setPin(String pin) { this.pin = pin; }

    public boolean verifyPin(String pin) {
        return this.pin.equals(pin);
    }

    public void closeAccount() {
        this.status = AccountStatus.CLOSED;
    }

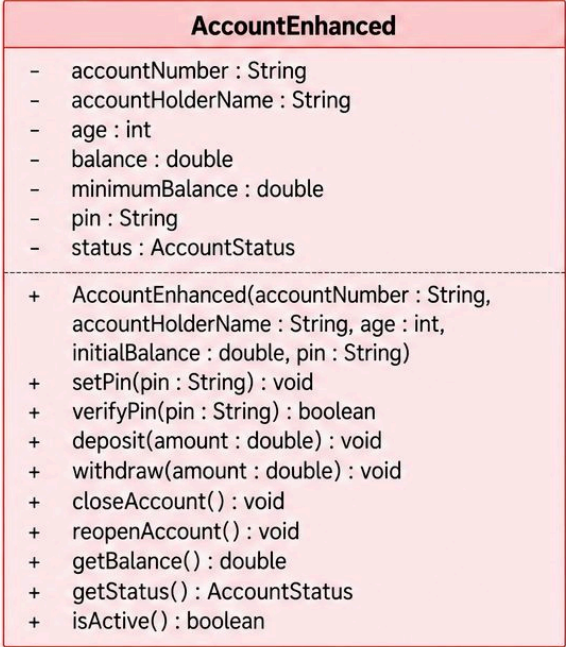
    public void deposit(double amount) {
        if (status != AccountStatus.ACTIVE)
            return; // blocked
        balance += amount;
    }
}
```



Activity 3 Class Diagram

Activity 3 – Enhanced Account Class (Class Diagram)

Objective: Add validation (age ≥ 18 , min balance), PIN protection, and status management to the Account class.



<<enumeration>>

AccountStatus

ACTIVE
INACTIVE
CLOSED

Key Points

- Validates age ≥ 18 when creating an account.
- Maintains a minimum balance.
- Adds PIN protection (set and verify PIN).
- Manages account status (ACTIVE, INACTIVE, CLOSED).
- Supports close and reopen account operations.
- Still supports deposit and withdraw operations from Activity 1.
- No Customer class – design focuses only on Account as per the activity plan.

Business Rules Introduced

- Account holder must be 18 years or older.
- Initial balance and subsequent balance must be $\geq$ minimum balance.
- PIN must be set and verified for sensitive operations (e.g., withdraw, close, reopen).
- Account can be in one of the statuses: ACTIVE, INACTIVE, CLOSED.
- Deposit and withdraw allowed only when account is ACTIVE.

Implementation Elements (from plan)

- AccountEnhanced class
- PIN field
- setPin()
- verifyPin()
- closeAccount()
- reopenAccount()

Design Concepts Demonstrated

- Validation
- State Management
- Business Rules
- Encapsulation
- Enum (AccountStatus)
- Constructors
- Building on previous implementation (Activity 1)

Next Steps

- Activity 4: Test validation, PIN verification, status transitions, and error scenarios.
- Activity 5: Replace boolean returns with meaningful exceptions for error conditions.

Testing Enhanced Account

Test validation, PIN verification, status transitions, and error scenarios with scenario-based test cases.

Key Concepts

- Testing
- Edge Cases
- Test Coverage

Implementation Elements

- TestAccountEnhanced
- scenario-based test cases
- expected output validation
- status transition tests
- PIN verification tests

Code Snapshot

```
public class TestAccountEnhanced {  
    public static void main(String[] args) {  
        AccountEnhanced acc = new  
AccountEnhanced("A001", "Bob", 1000.0, 25);  
        acc.setPin("1234");  
  
        // Test PIN verification  
        System.out.println(acc.verifyPin("1234")); // true  
        System.out.println(acc.verifyPin("0000")); // false  
  
        // Test status transition  
        acc.closeAccount();  
        acc.deposit(500); // should be blocked  
        System.out.println("Balance: " + acc.getBalance()); //  
unchanged  
    }  
}
```

Valid PIN

Invalid PIN

Deposit on Active
Account

Deposit on Closed
Account

Exceptions in Account Class

Replace boolean returns with meaningful custom exceptions for all error conditions in the Account class.

Key Concepts

- Exceptions
- Custom Exceptions
- Error Handling

Implementation Elements

- AccountException
- InvalidAmountException
- InsufficientBalanceException
- MinimumBalanceViolationException
- InactiveAccountException
- InvalidPinException

Code Snapshot

```
public class InsufficientBalanceException extends
AccountException {
    public InsufficientBalanceException(double balance,
double amount) {
        super("Insufficient balance: " + balance + " for
withdrawal of " + amount);
    }
}

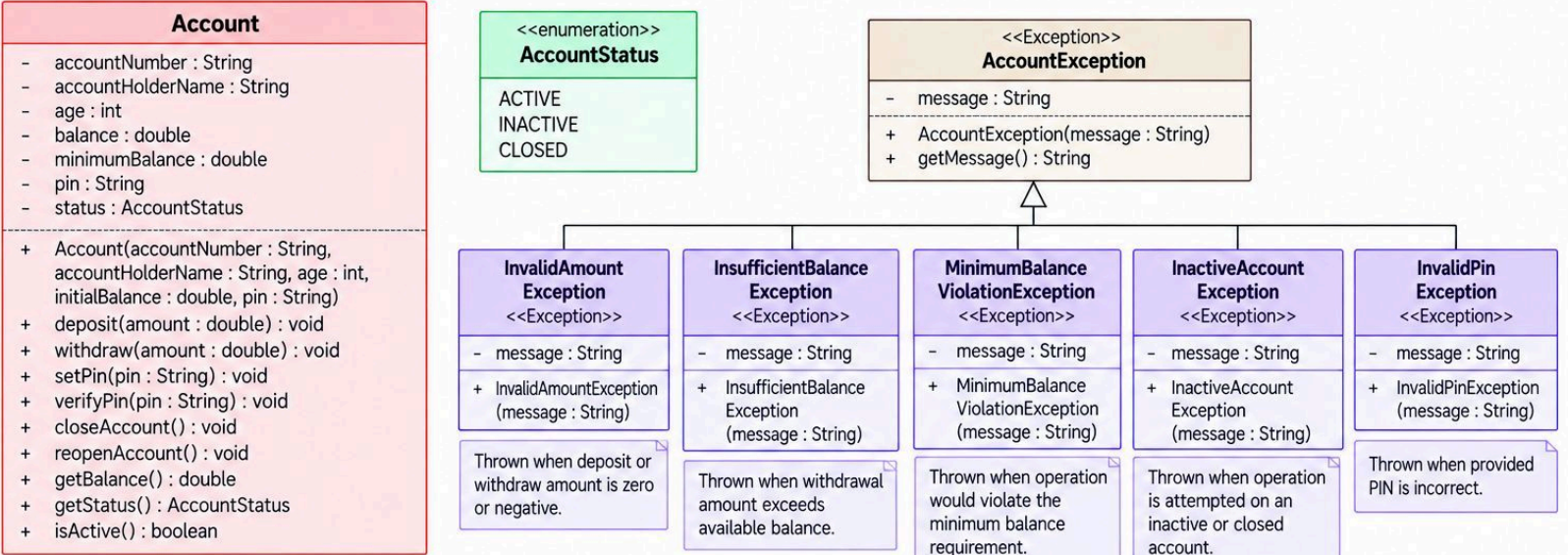
public void withdraw(double amount) throws
InsufficientBalanceException,
    MinimumBalanceViolationException {
    if (amount > balance)
        throw new InsufficientBalanceException(balance,
amount);
    if (balance - amount < minBalance)
        throw new
MinimumBalanceViolationException(balance, amount,
minBalance);
    balance -= amount;
}
```

AccountException Parent exception for all account-related errors.	InvalidAmountException Thrown for invalid deposit or withdrawal amounts.	InsufficientBalanceException Thrown when withdrawal exceeds current balance.	MinimumBalanceViolationException Thrown when an operation would drop balance below minimum.
InactiveAccountException Thrown when an operation is attempted on an inactive account.		InvalidPinException Thrown when an incorrect PIN is provided.	

Activity 5 Class Diagram

Activity 5 – Exceptions in Account Class (Class Diagram)

Objective: Replace boolean returns with meaningful exceptions for error conditions.



Key Changes from Previous Activities

- Replaced boolean returns with meaningful exceptions.
- Introduced custom exception classes for different error conditions.
- Account methods (deposit, withdraw, verifyPin, etc.) now throw exceptions instead of returning false.
- More expressive error handling aligned with real-world banking systems.
- Builds on validation, PIN protection, and status management from Activity 3.

Key Tech Concepts (Tags)

- Exceptions
- Custom Exceptions
- Error Handling
- Throw and Catch
- Checked vs Unchecked Exceptions
- Clean API Design
- Domain-specific Error Handling

Implementation Elements (from plan)

- AccountException (base exception)
- InvalidAmountException
- InsufficientBalanceException
- MinimumBalanceViolationException
- InactiveAccountException
- InvalidPinException

Example Usage in Account Methods

```
public void withdraw(double amount) {
    if (!isActive())
        throw new InactiveAccountException(
            "Account is not active");
    if (amount <= 0)
        throw new InvalidAmountException(
            "Amount must be greater than zero");
    if (balance - amount < minimumBalance)
        throw new MinimumBalanceViolationException(
            "Cannot withdraw. Minimum balance required.");
    if (balance < amount)
        throw new InsufficientBalanceException(
            "Insufficient balance.");
    balance -= amount;
}
```

Testing Exceptions

Write test code using try-catch blocks to verify all custom exception scenarios are correctly thrown with meaningful messages.

Key Concepts

- Try-Catch
- Exception Handling
- Error Reporting

Implementation Elements

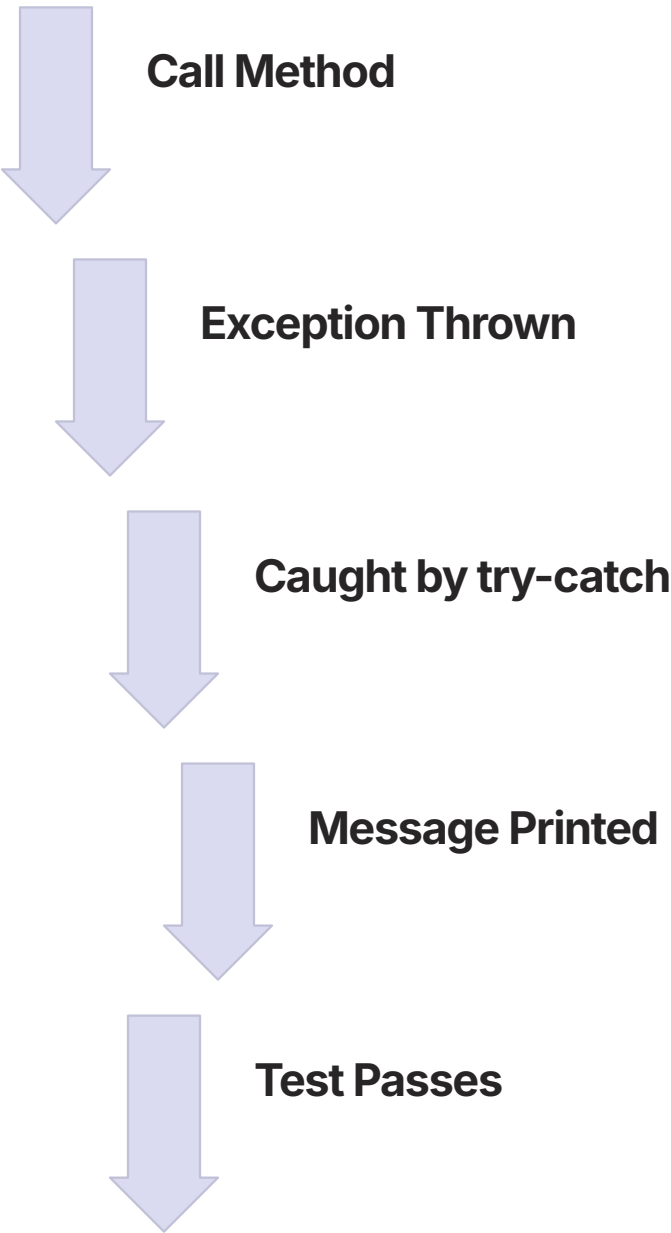
- TestAccountExceptions
- try-catch blocks
- exception message validation
- multiple exception scenarios

Code Snapshot

```
public class TestAccountExceptions {
    public static void main(String[] args) {
        Account acc = new Account("A001", "Alice", 600.0);

        try {
            acc.withdraw(700.0);
        } catch (InsufficientBalanceException e) {
            System.out.println("Caught: " + e.getMessage());
        }

        try {
            acc.withdraw(200.0); // drops below min balance
        } catch (MinimumBalanceViolationException e) {
            System.out.println("Caught: " + e.getMessage());
        }
    }
}
```



Account Subclasses

Create SavingsAccount and CurrentAccount subclasses extending the base Account with type-specific behavior.

Key Concepts

- Inheritance
- Extends
- Super
- Method Overriding

Implementation Elements

- SavingsAccount
- CurrentAccount
- super()
- overridden methods
- interestRate field
- overdraftLimit field
- specialized fields

Code Snapshot

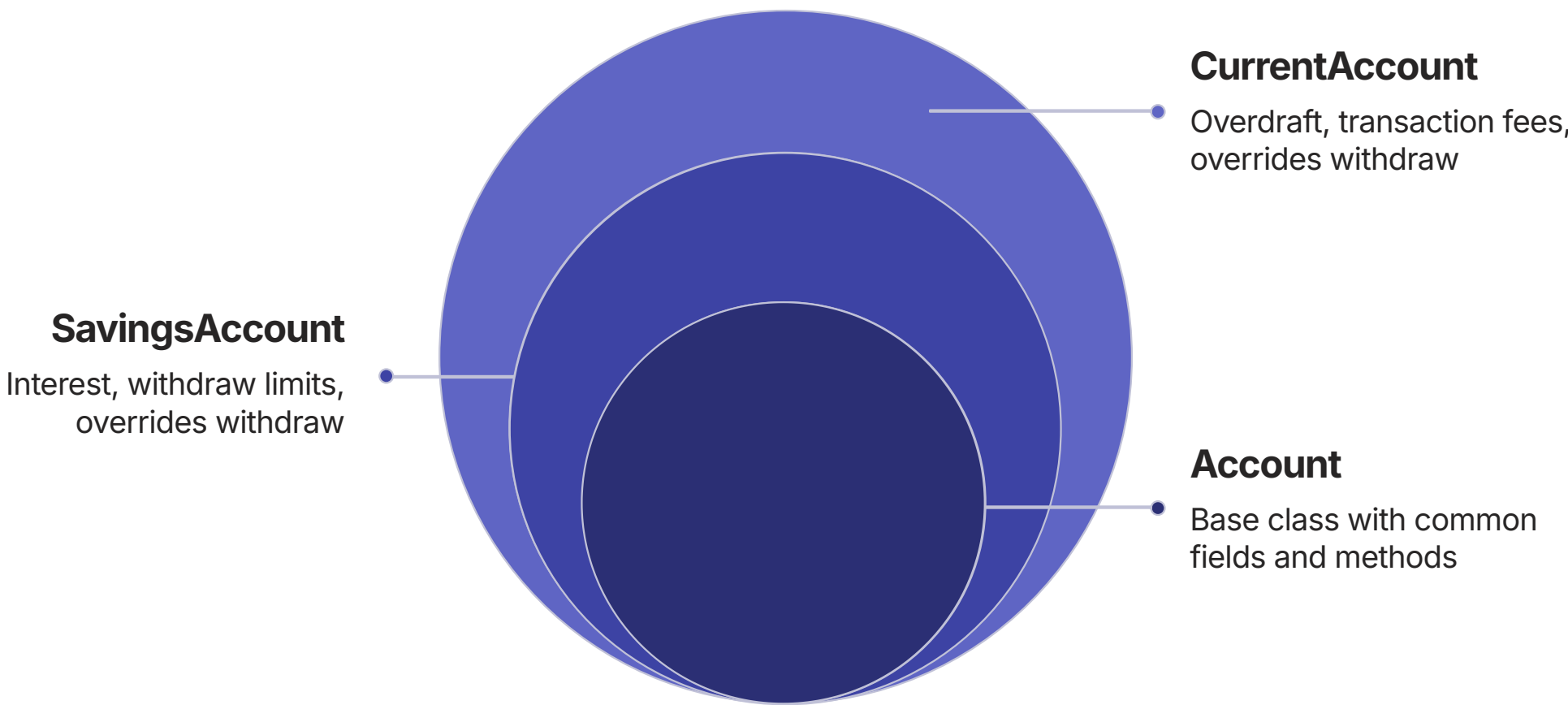
```
public class SavingsAccount extends Account {
    private double interestRate;

    public SavingsAccount(String accNo, String name,
double balance, double rate) {
        super(accNo, name, balance);
        this.interestRate = rate;
    }

    public void applyInterest() {
        balance += balance * interestRate / 100;
    }
}

public class CurrentAccount extends Account {
    private double overdraftLimit;

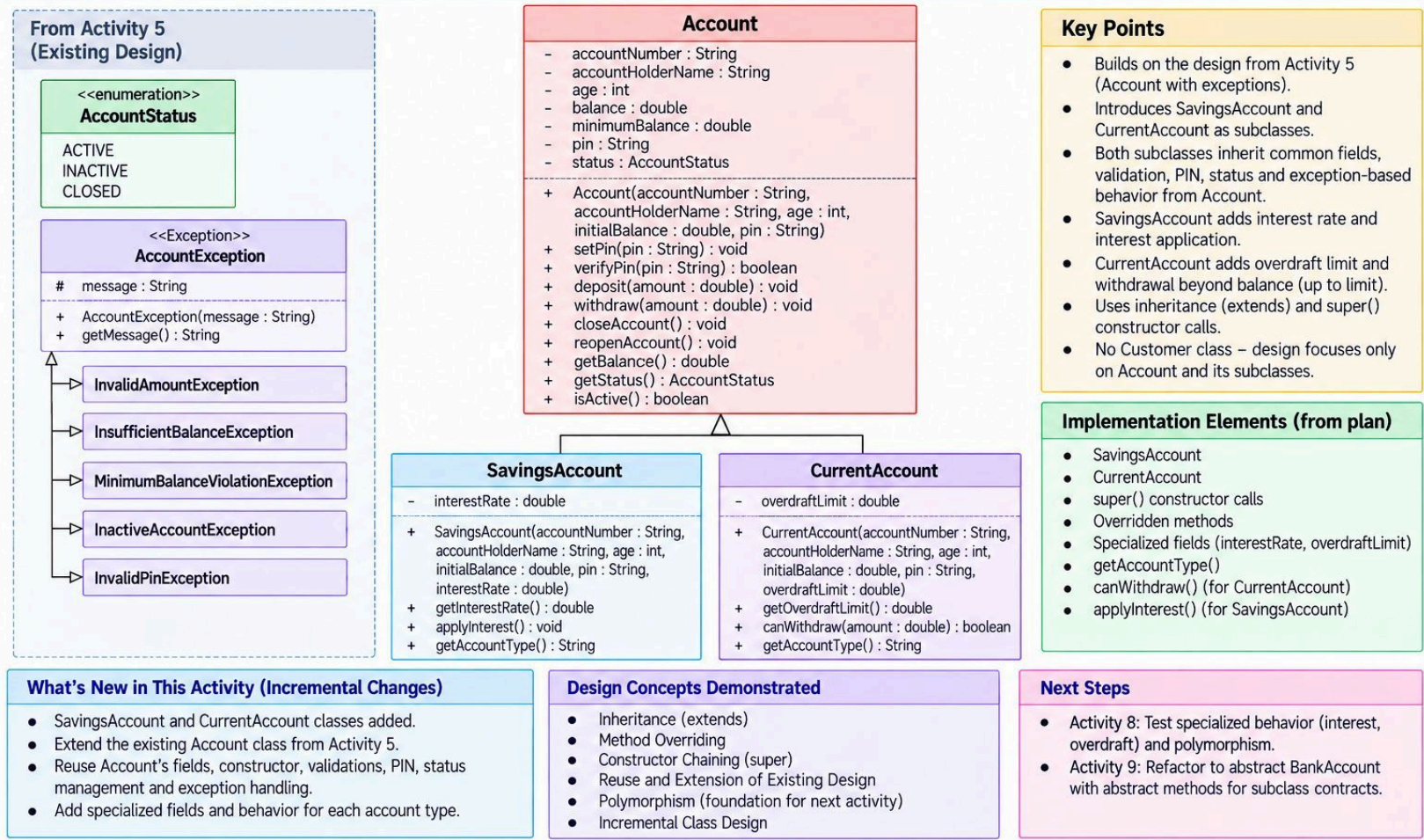
    @Override
    public void withdraw(double amount) throws
InsufficientBalanceException {
        if (amount > balance + overdraftLimit)
            throw new InsufficientBalanceException(balance,
amount);
        balance -= amount;
    }
}
```



Activity 7 Class Diagram

Activity 7 – Account Subclasses (Class Diagram)

Objective: Create SavingsAccount and CurrentAccount extending the base Account class.



Testing Subclasses

Test specialized behavior (interest calculation, overdraft) and polymorphism across account types.

Key Concepts

- Polymorphism
- Dynamic Dispatch
- Type-Specific Testing

Implementation Elements

- TestAccountSubclasses
- polymorphic references
- type-specific method tests
- applyInterest() test
- overdraft test

Code Snapshot

```
public class TestAccountSubclasses {  
    public static void main(String[] args) {  
        Account[] accounts = {  
            new SavingsAccount("S001", "Alice", 1000.0, 5.0),  
            new CurrentAccount("C001", "Bob", 500.0, 200.0)  
        };  
  
        for (Account acc : accounts) {  
            System.out.println(acc.getAccountType() + ": " +  
acc.getBalance());  
        }  
  
        SavingsAccount sa = (SavingsAccount) accounts[0];  
        sa.applyInterest();  
        System.out.println("After interest: " + sa.getBalance());  
    }  
}
```

SavingsAccount

Apply interest test

CurrentAccount

Overdraft test

Abstract BankAccount

Refactor to an abstract BankAccount class with abstract methods that enforce contracts on all subclasses.

Key Concepts

- Abstract Class
- Abstract Methods
- Template Method
- Contract

Implementation Elements

- BankAccount (abstract)
- getMinimumBalance()
- getAccountType()
- getInterestRate()
- canWithdraw()
- applyMonthlyInterest()

Code Snapshot

```
public abstract class BankAccount {
    protected double balance;

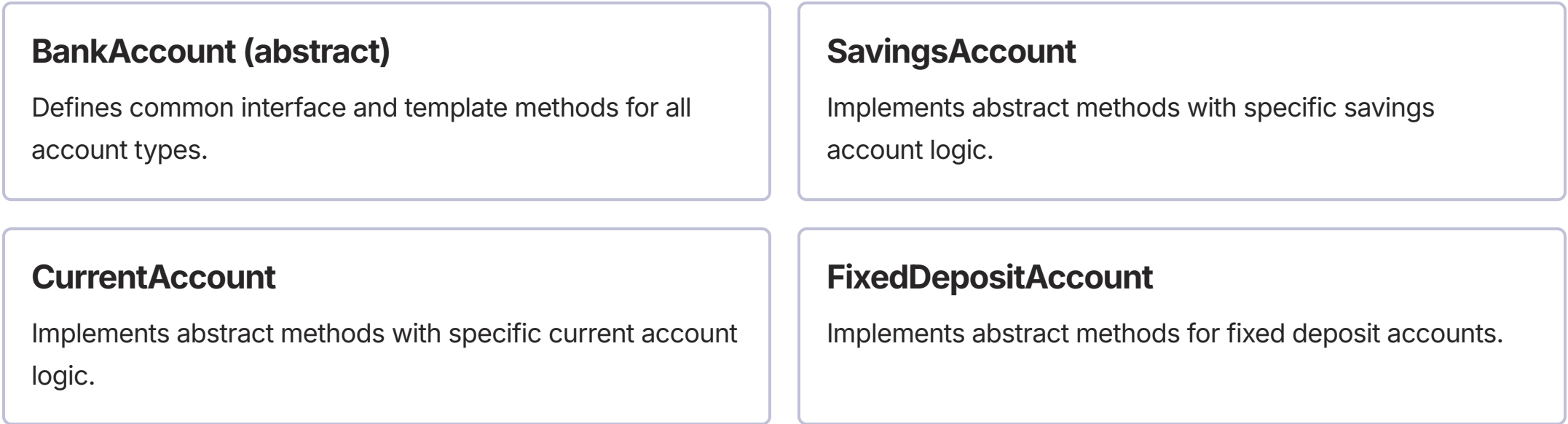
    public abstract double getMinimumBalance();
    public abstract String getAccountType();
    public abstract double getInterestRate();

    public boolean canWithdraw(double amount) {
        return (balance - amount) >= getMinimumBalance();
    }

    public void applyMonthlyInterest() {
        balance += balance * getInterestRate() / 100;
    }

    public void withdraw(double amount) throws Exception
    {
        if (!canWithdraw(amount))
            throw new
            MinimumBalanceViolationException(balance, amount,
            getMinimumBalance());
        balance -= amount;
    }
}
```

Below is the class hierarchy for the abstract BankAccount and its subclasses.



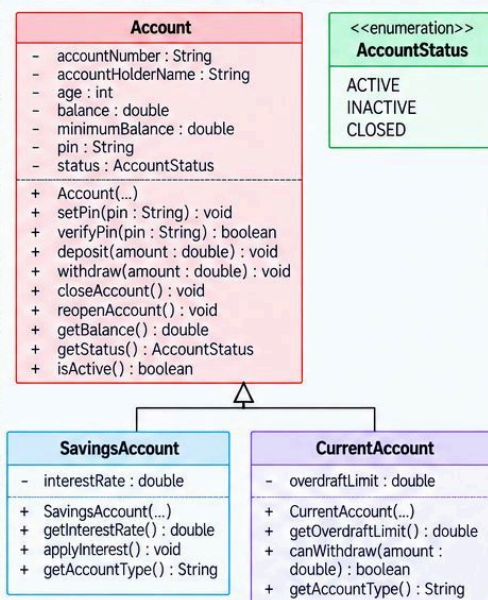
Activity 9 Class Diagram

Activity 9 – Abstract BankAccount (Class Diagram)

Objective: Refactor to abstract BankAccount with abstract methods for subclass contracts.

From Activity 7 (Existing Design)

Account with subclasses (no changes here)

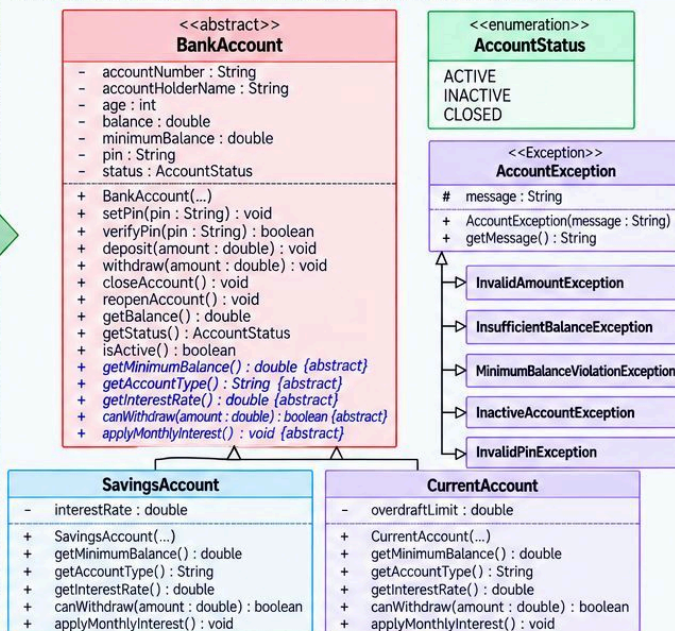


Refactor to
Abstract Class

Activity 9

(Refactored Design with Abstract BankAccount)

Account is now abstract with abstract methods. Subclasses implement the contract.



Key Points

- Refactors Account to abstract BankAccount.
- Defines abstract methods for subclass contracts.
- SavingsAccount and CurrentAccount implement the abstract methods.
- Common fields, validation, PIN, status and exception handling remain in BankAccount.
- Enforces type-specific rules for minimum balance, interest rate and withdrawal logic.
- Builds on the design from Activity 7 (inheritance) and retains exceptions from Activity 5.
- No Customer class – design focuses only on Account and its subclasses.

What's New in This Activity (Incremental Changes)

- Replaces concrete Account with abstract BankAccount.
- Introduces abstract methods for subclass contract.
- SavingsAccount and CurrentAccount implement these methods.
- Keeps existing fields, constructors, validation, PIN, status and exception handling.
- Builds on Activity 7 and uses exceptions from Activity 5.

Design Concepts Demonstrated

- Abstract Class
- Abstract Methods
- Inheritance and Method Implementation
- Template Method (common functionality in base class)
- Polymorphism (preparation for Activity 10)
- Incremental Refactoring
- Reuse of Existing Design
- Domain Modeling with Contracts

Implementation Elements (from plan)

- BankAccount (abstract class)
- Abstract methods: `getMinimumBalance()`, `getAccountType()`, `getInterestRate()`, `canWithdraw()`, `applyMonthlyInterest()`
- SavingsAccount (implements abstract methods)
- CurrentAccount (implements abstract methods)
- AccountStatus enum (from previous activities)
- Exception classes (from Activity 5)

Next Steps

- Activity 10: Test abstraction, polymorphism, and type-specific rules across all account types.
- Activity 11: Introduce `IAccount` interface and `AccountFactory`.

Testing Abstract Class

Test abstraction, polymorphism, and type-specific rules across all account types using an accounts array.

Key Concepts

- Polymorphism
- Abstract Dispatch
- Type-Specific Rules

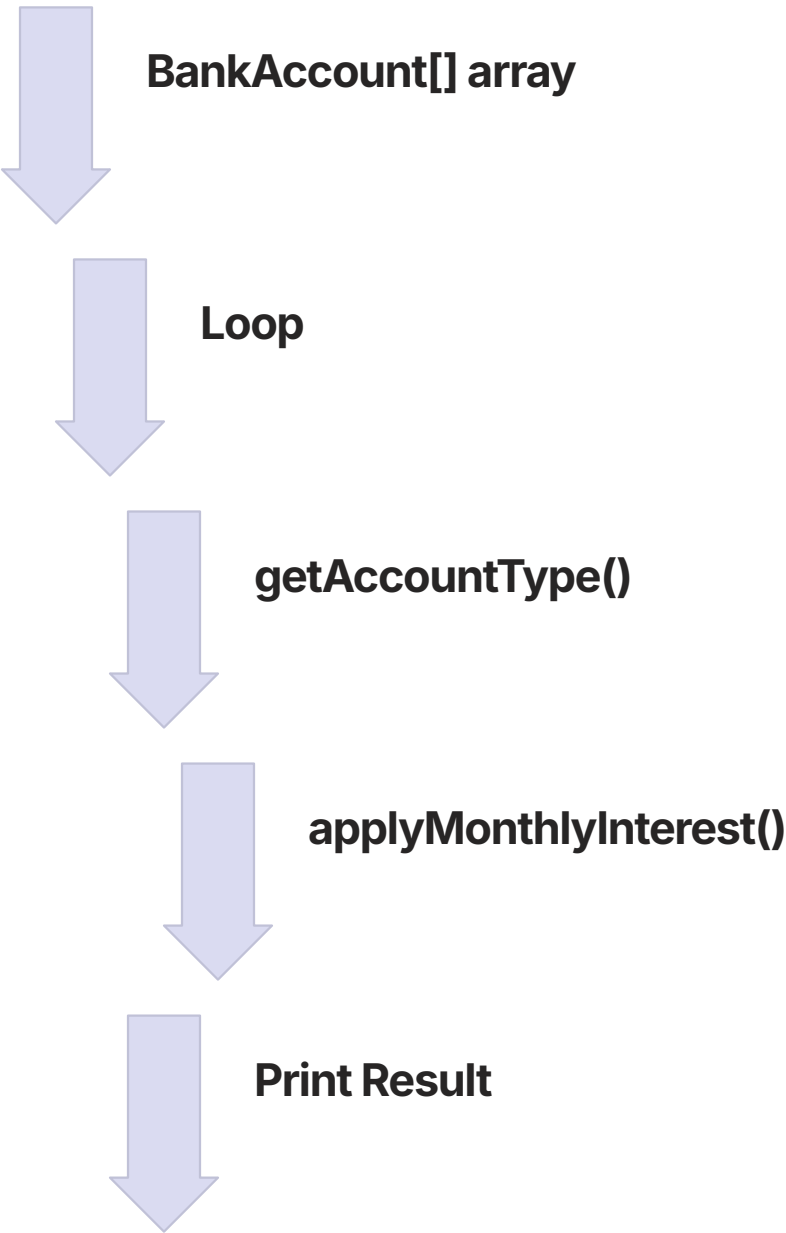
Implementation Elements

- TestAbstractAccount
- accounts array
- dynamic method invocation
- applyMonthlyInterest() across types

Code Snapshot

```
public class TestAbstractAccount {
    public static void main(String[] args) {
        BankAccount[] accounts = {
            new SavingsAccount("S001", "Alice", 1000.0),
            new CurrentAccount("C001", "Bob", 2000.0),
            new FixedDepositAccount("F001", "Carol", 5000.0)
        };

        for (BankAccount acc : accounts) {
            System.out.println(acc.getAccountType()
                + " | Min Balance: " + acc.getMinimumBalance()
                + " | Rate: " + acc.getInterestRate());
            acc.applyMonthlyInterest();
            System.out.println("After interest: " +
                acc.getBalance());
        }
    }
}
```



IAccount Interface & Factory

Introduce IAccount interface, AbstractAccount, and AccountFactory for flexible, loosely-coupled account creation across four account types.

Key Concepts

- Interface
- Factory Pattern
- Loose Coupling
- Abstraction

Implementation Elements

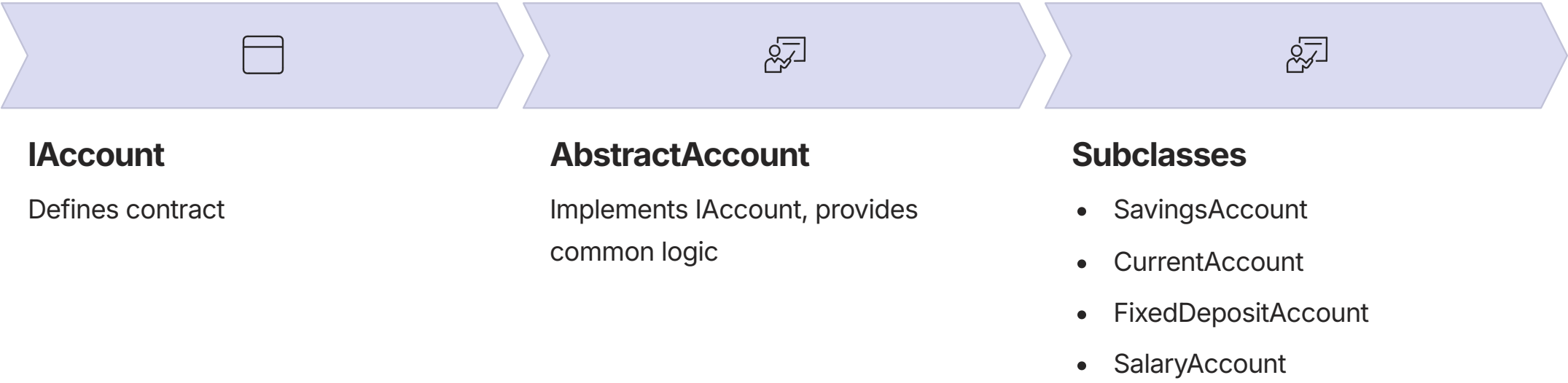
- IAccount
- AbstractAccount
- AccountFactory
- SavingsAccount
- CurrentAccount
- FixedDepositAccount
- SalaryAccount

Code Snapshot

```
public interface IAccount {
    void deposit(double amount) throws AccountException;
    void withdraw(double amount) throws
AccountException;
    double getBalance();
    String getAccountType();
}

public class AccountFactory {
    public static IAccount createAccount(String type, String
accNo,
        String name, double balance) {
        return switch (type) {
            case "SAVINGS" -> new SavingsAccount(accNo,
name, balance);
            case "CURRENT" -> new CurrentAccount(accNo,
name, balance);
            case "FIXED"  -> new FixedDepositAccount(accNo,
name, balance);
            case "SALARY" -> new SalaryAccount(accNo, name,
balance);
            default -> throw new
IllegalArgumentException("Unknown type: " + type);
        };
    }
}
```

The diagram below illustrates the relationships between the IAccount interface, AbstractAccount, its concrete subclasses, and the AccountFactory.



AccountFactory
Creates instances of IAccount implementations

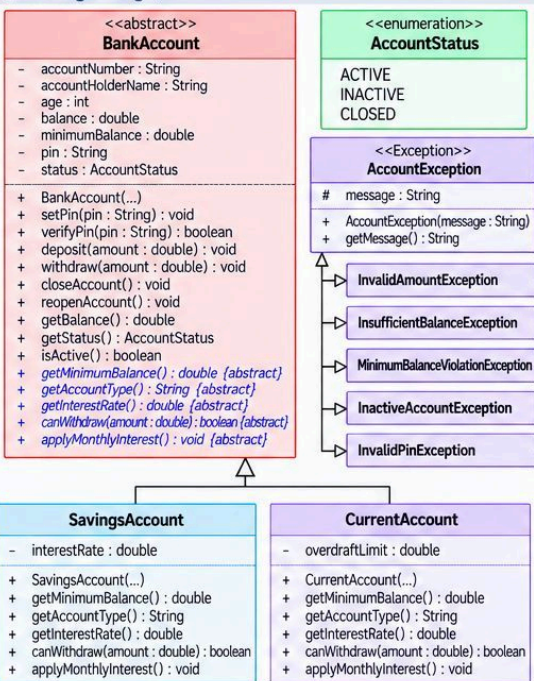
IAccount (Output)
Returns the interface type, ensuring loose coupling

Activity 11 Class Diagram

Activity 11 – IAccount Interface & Factory (Class Diagram)

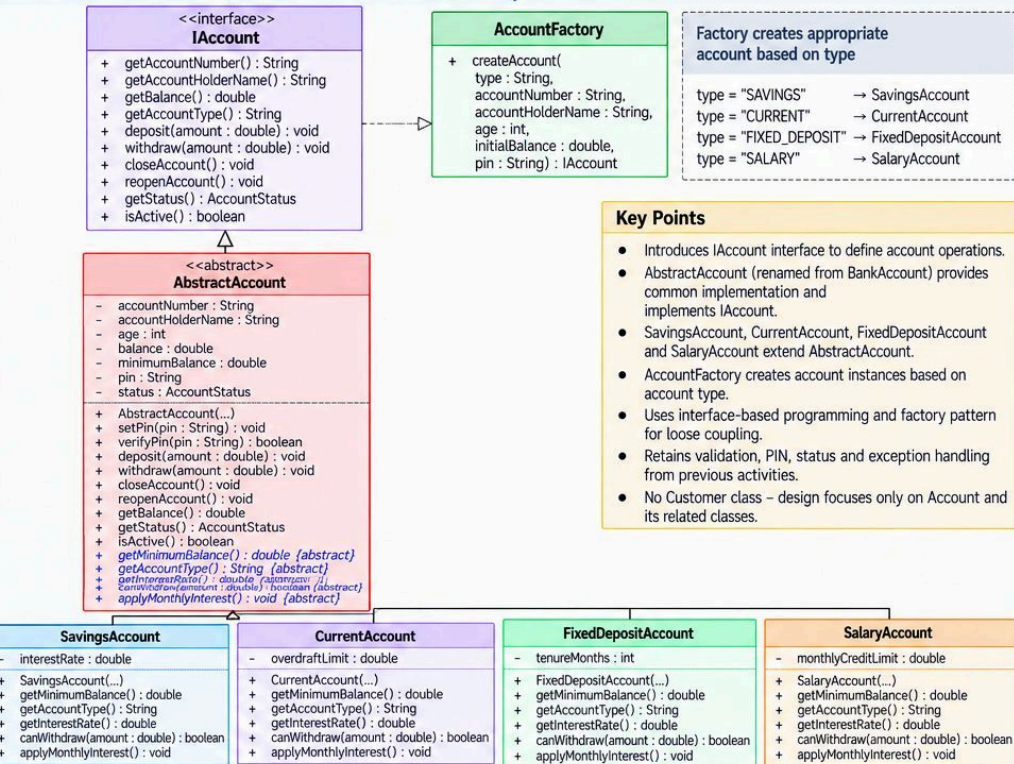
Objective: Introduce IAccount interface, AbstractAccount, and AccountFactory for creation.

From Activity 9 (Existing Design)



Activity 11 Design (Incremental)

Introduces IAccount interface, AbstractAccount and AccountFactory.



What's New in This Activity (Incremental Changes)

- Introduces IAccount interface.
- Refactors BankAccount to AbstractAccount that implements IAccount.
- Adds FixedDepositAccount and SalaryAccount classes.
- Introduces AccountFactory for creating account instances.
- Keeps existing fields, validation, PIN, status and exception handling.
- Builds on design from previous activities (Activities 5-9).

Design Concepts Demonstrated

- Interface
- Abstract Class
- Inheritance (extends)
- Factory Pattern
- Polymorphism
- Loose Coupling
- Static Factory Method

Implementation Elements (from plan)

- IAccount interface
- AbstractAccount class
- AccountFactory class
- SavingsAccount, CurrentAccount, FixedDepositAccount, SalaryAccount
- createAccount() factory method: method
- super() constructor calls
- Overridden methods
- Type-specific fields and behavior

Next Steps

- Activity 12: Test interface-based operations and factory-based account creation.
- Activity 13: Introduce rules engine for tenure-based rules.

Testing Interface & Factory

Test interface-based operations and factory-based account creation using runtime polymorphism.

Key Concepts

- Interface Polymorphism
- Factory Usage
- Runtime Selection

Implementation Elements

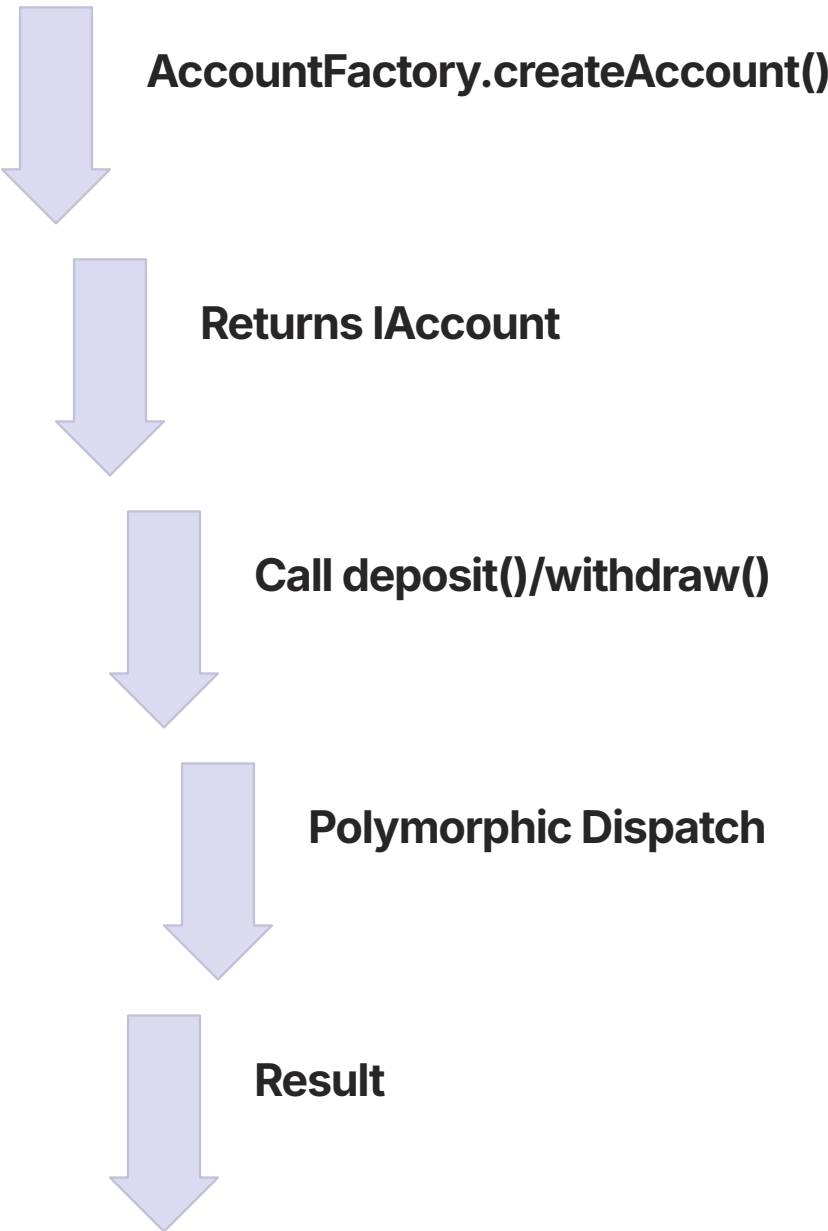
- TestInterfaceFactory
- factory-driven creation
- interface references
- runtime type selection

Code Snapshot

```
public class TestInterfaceFactory {
    public static void main(String[] args) throws
    AccountException {
        IAccount savings =
        AccountFactory.createAccount("SAVINGS", "S001", "Alice",
        1000.0);
        IAccount current =
        AccountFactory.createAccount("CURRENT", "C001", "Bob",
        2000.0);
        IAccount fixed =
        AccountFactory.createAccount("FIXED", "F001", "Carol",
        5000.0);

        savings.deposit(500.0);
        System.out.println(savings.getAccountType() + ": " +
        savings.getBalance());

        current.withdraw(300.0);
        System.out.println(current.getAccountType() + ": " +
        current.getBalance());
    }
}
```



Rules Engine – If-Else

Build AccountRulesEngine using if-else logic for tenure-based rules covering minimum balance, interest rate, and feature names.

Key Concepts

- Singleton
- Conditional Logic
- Tenure Buckets
- Real Bank Data

Implementation Elements

- AccountRulesEngine (Singleton)
- getMinimumBalance()
- getInterestRate()
- getFeatureName()
- getTenureBucketName()
- tenure-based if-else chains

Code Snapshot

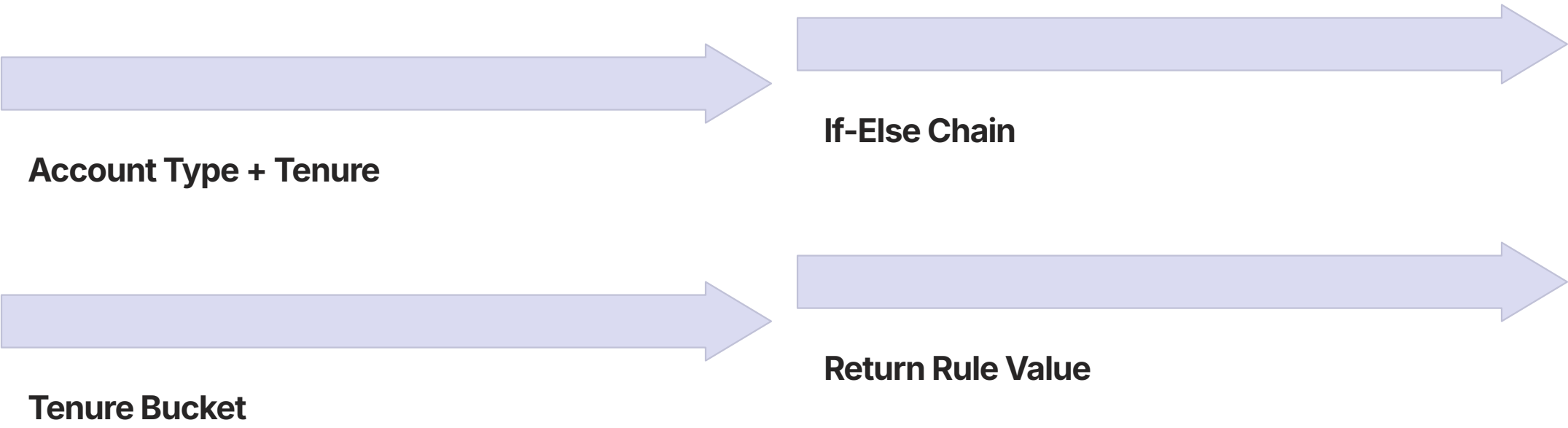
```
public class AccountRulesEngine {
    private static AccountRulesEngine instance;

    private AccountRulesEngine() {}

    public static AccountRulesEngine getInstance() {
        if (instance == null) instance = new
AccountRulesEngine();
        return instance;
    }

    public double getInterestRate(String type, int tenure) {
        if (type.equals("SAVINGS")) {
            if (tenure < 1) return 2.5;
            else if (tenure < 3) return 3.5;
            else return 4.5;
        } else if (type.equals("FIXED")) {
            if (tenure < 1) return 5.0;
            else return 6.5;
        }
        return 0.0;
    }
}
```

Below, a stepsWithArrow diagram: Account Type + Tenure → If-Else Chain → Tenure Bucket → Return Rule Value



Rules Engine – Map-Based

Refactor the Rules Engine to use a nested Map with an inner Rule class for O(1) lookup, replacing if-else chains.

Key Concepts

- Map Data Structure
- Inner Class
- O(1) Lookup
- Refactoring

Implementation Elements

- AccountRulesEngine.Rule (inner class)
- Map<String, Map<Integer, Rule>>
- loadRules()
- rule lookup by type and tenure

Code Snapshot

```
public class AccountRulesEngine {
    private Map<String, Map<Integer, Rule>> rules = new
    HashMap<>();

    private static class Rule {
        double interestRate;
        double minBalance;
        String featureName;

        Rule(double rate, double minBal, String feature) {
            this.interestRate = rate;
            this.minBalance = minBal;
            this.featureName = feature;
        }
    }

    private void loadRules() {
        Map<Integer, Rule> savingsRules = new HashMap<>();
        savingsRules.put(0, new Rule(2.5, 500, "Basic
Savings"));
        savingsRules.put(1, new Rule(3.5, 1000, "Regular
Savings"));
        savingsRules.put(3, new Rule(4.5, 2000, "Premium
Savings"));
        rules.put("SAVINGS", savingsRules);
    }
}
```

Below, a solidBoxes diagram comparing: If-Else (O(n) lookup, hard to maintain) vs Map-Based (O(1) lookup, easy to extend)

If-Else
O(n) lookup, hard to maintain

Map-Based
O(1) lookup, easy to extend

Rules Engine – Properties

Move rules to .properties files, load dynamically at startup, and support hot reload without restarting the application.

Key Concepts

- Properties Files
- Externalized Config
- Resource Loading
- Hot Reload

Implementation Elements

- AccountRulesPropertiesLoader
- savings.properties
- current.properties
- fixeddeposit.properties
- salary.properties
- reloadRules()

Code Snapshot

```
// savings.properties
savings.tenure.0.interestRate=2.5
savings.tenure.0.minBalance=500
savings.tenure.1.interestRate=3.5
savings.tenure.1.minBalance=1000

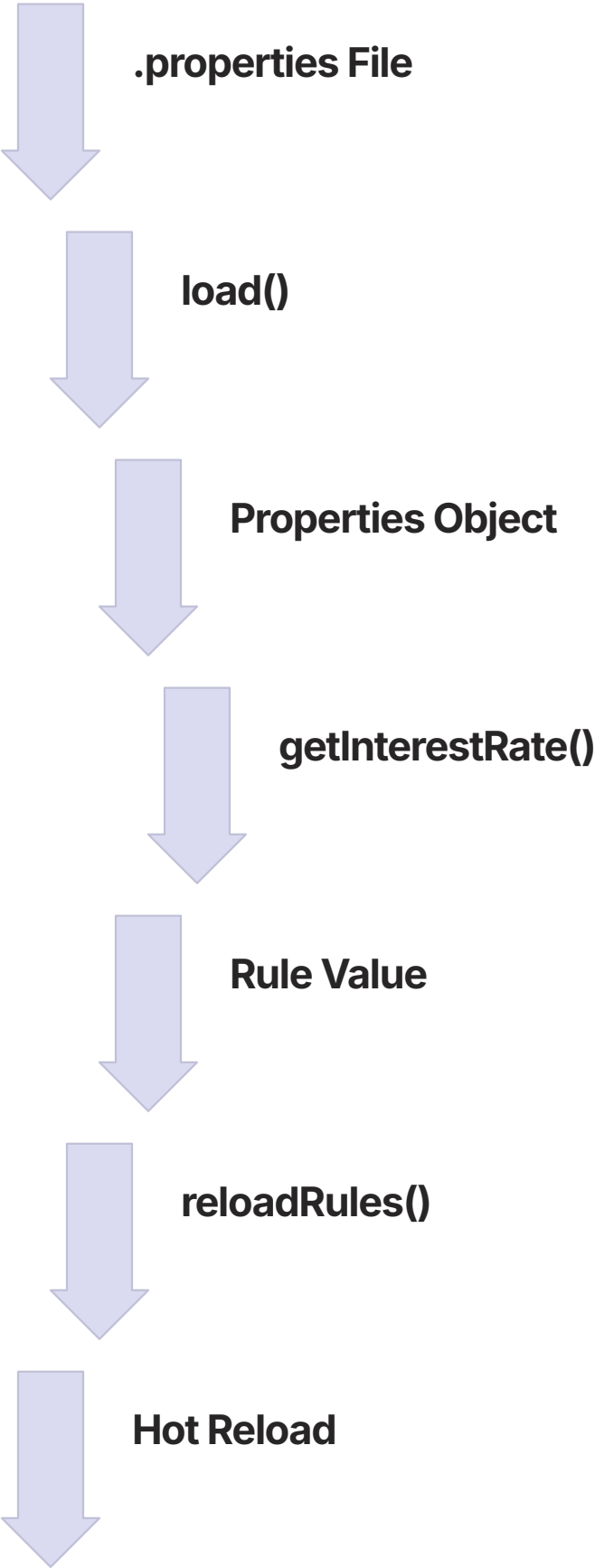
// AccountRulesPropertiesLoader.java
public class AccountRulesPropertiesLoader {
    private Properties props = new Properties();

    public void load(String filename) throws IOException {
        try (InputStream is =
getClass().getResourceAsStream(filename)) {
            props.load(is);
        }
    }

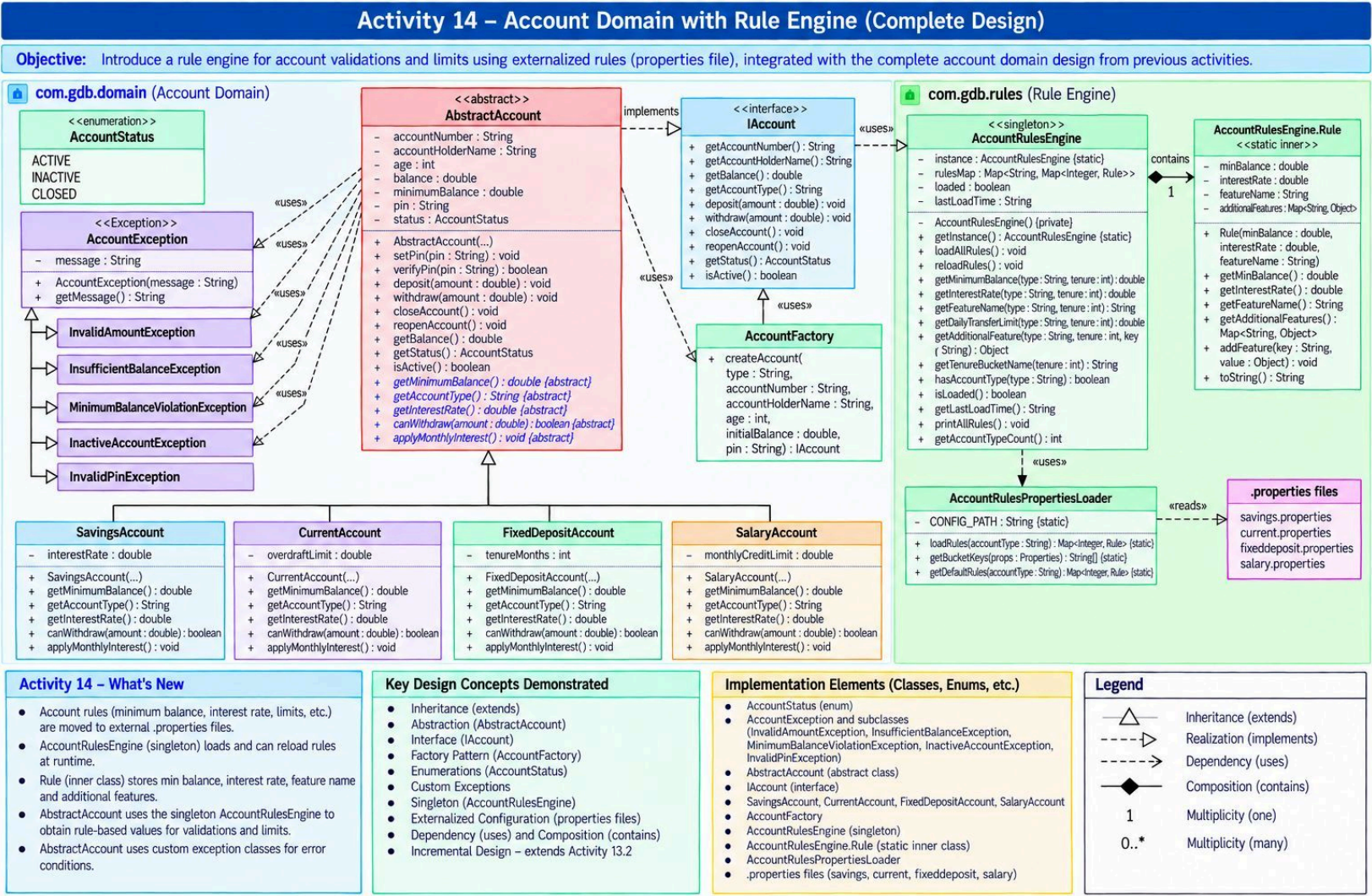
    public double getInterestRate(String type, int tenure) {
        String key = type.toLowerCase() + ".tenure." + tenure +
".interestRate";
        return Double.parseDouble(props.getProperty(key,
"0.0"));
    }

    public void reloadRules() throws IOException {
        load("savings.properties");
        load("current.properties");
    }
}
```

Below, a stepsWithArrow flow: .properties File → load() → Properties Object → getInterestRate() → Rule Value → reloadRules() → Hot Reload



Activity 14 Class Diagram



Transfer with Daily Limits

Implement TransferService with privilege-based daily transfer limits, tracking daily totals and resetting on date change.

Key Concepts

- Service Layer
- Atomic Operations
- Daily Limits
- Privilege Rules

Implementation Elements

- TransferService
- dailyTransferTotal
- lastTransferDate
- getDailyTransferLimit()
- canTransfer()
- properties: daily.transfer.limit.STANDARD, daily.transfer.limit.PREMIUM

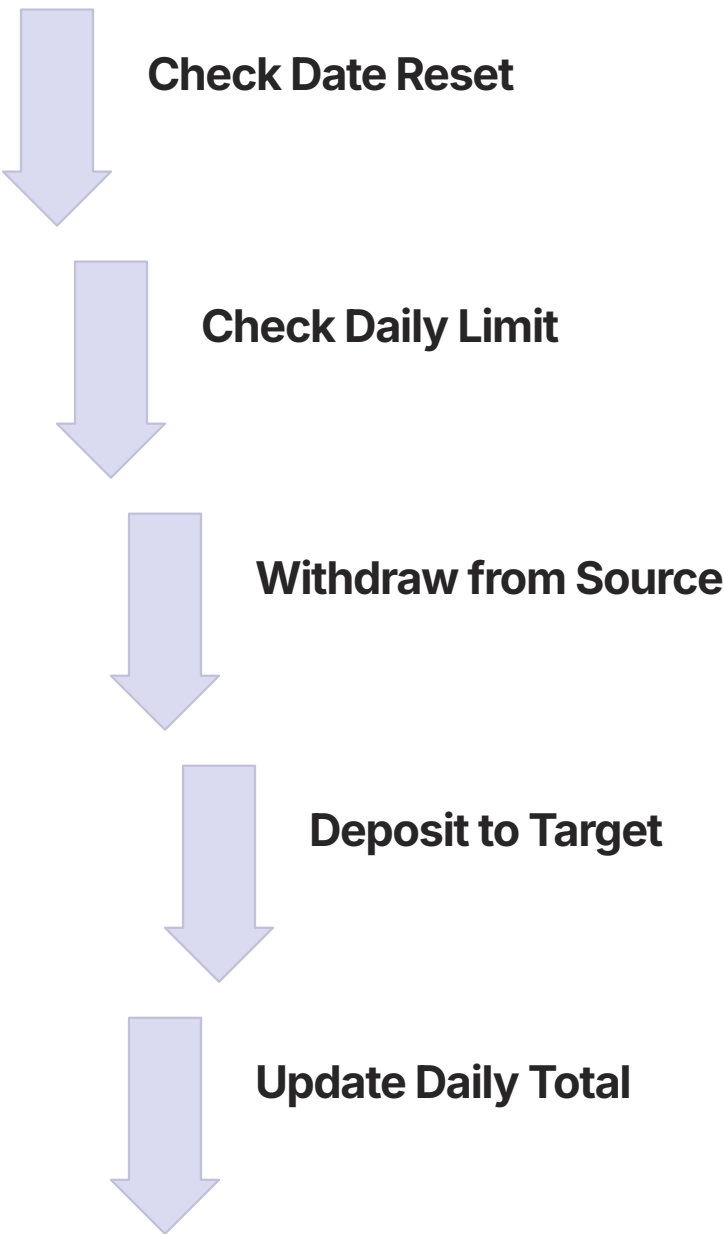
Code Snapshot

```
public class TransferService {
    private double dailyTransferTotal = 0;
    private LocalDate lastTransferDate = LocalDate.now();

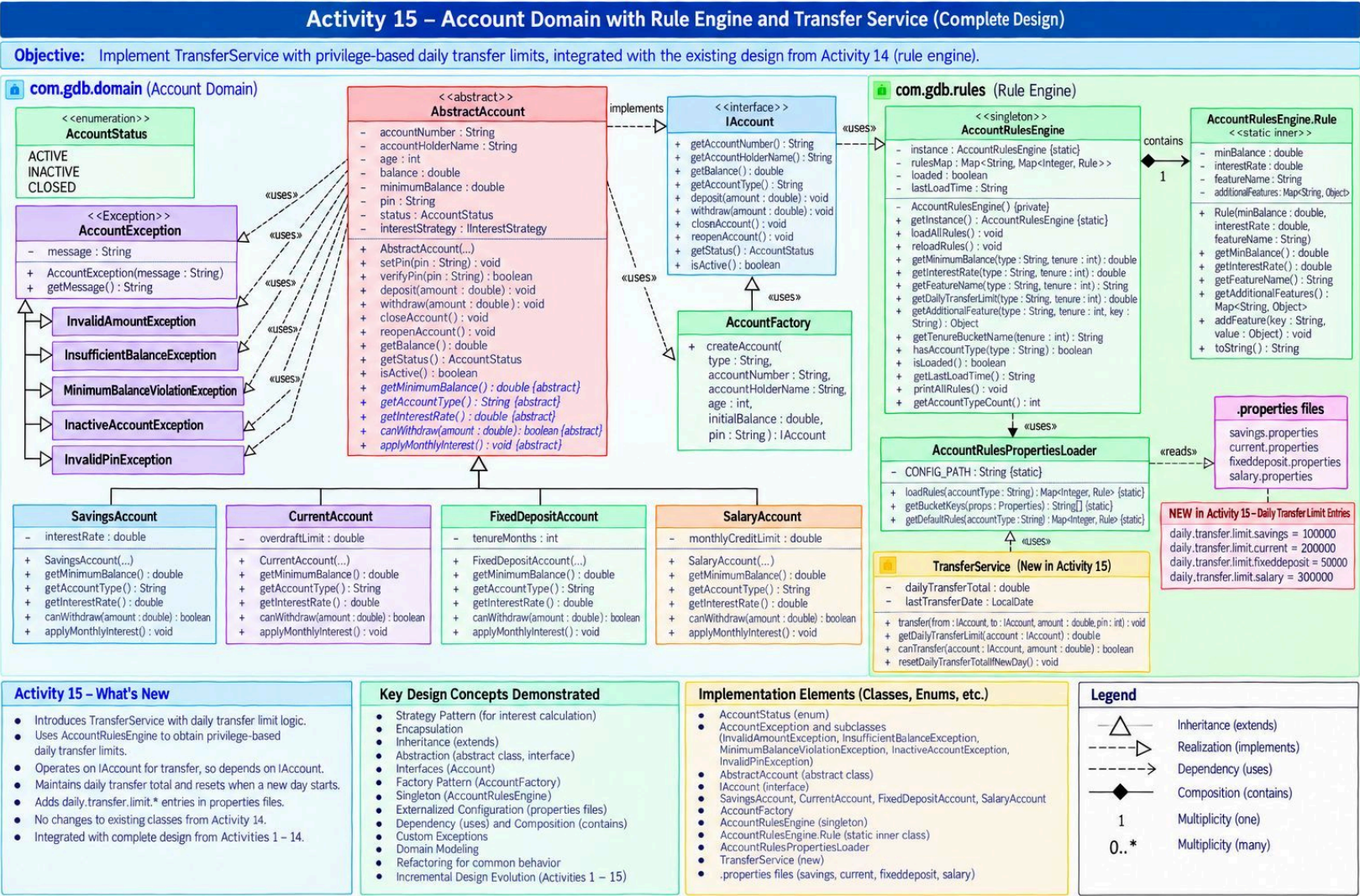
    public boolean canTransfer(double amount, String
privilege) {
        if (!LocalDate.now().equals(lastTransferDate)) {
            dailyTransferTotal = 0;
            lastTransferDate = LocalDate.now();
        }
        double limit = getDailyTransferLimit(privilege);
        return (dailyTransferTotal + amount) <= limit;
    }

    public void transfer(IAccount from, IAccount to, double
amount, String privilege)
        throws AccountException {
        if (!canTransfer(amount, privilege))
            throw new DailyLimitExceededException(amount,
getDailyTransferLimit(privilege));
        from.withdraw(amount);
        to.deposit(amount);
        dailyTransferTotal += amount;
    }
}
```

Below, a stepsWithArrow flow: Check Date Reset → Check Daily Limit → Withdraw from Source → Deposit to Target → Update Daily Total



Activity 15 Class Diagram



Transaction Model

Introduce a Transaction class and TransactionType enum; overload account methods to return Transaction objects alongside existing void methods.

Key Concepts

- Domain Modeling
- Enums
- Method Overloading
- Coexistence

Implementation Elements

- Transaction class
- TransactionType enum (DEPOSIT, WITHDRAW, TRANSFER)
- timestamp
- amount
- accountNumber
- new overloaded methods returning Transaction

Code Snapshot

```
public enum TransactionType { DEPOSIT, WITHDRAW, TRANSFER }

public class Transaction {
    private String transactionId;
    private TransactionType type;
    private double amount;
    private LocalDateTime timestamp;
    private String accountNumber;

    public Transaction(TransactionType type, double amount, String accNo) {
        this.transactionId = UUID.randomUUID().toString();
        this.type = type;
        this.amount = amount;
        this.accountNumber = accNo;
        this.timestamp = LocalDateTime.now();
    }
}

// Overloaded method in Account:
public Transaction depositWithTransaction(double amount) throws AccountException {
    deposit(amount); // existing method
    return new Transaction(TransactionType.DEPOSIT, amount, accountNumber);
}
```

Below, a solidBoxes diagram showing Transaction fields: transactionId, type (enum), amount, timestamp, accountNumber



Command Pattern + File Logging

Wrap Deposit, Withdraw, and Transfer operations as Command objects and serialize them to a file for persistent transaction logging.

Key Concepts

- Command Pattern
- Java Serialization
- File I/O
- Encapsulation

Implementation Elements

- TransactionCommand (interface)
- DepositCommand
- WithdrawCommand
- TransferCommand
- TransactionLog
- ObjectOutputStream
- ObjectInputStream

Code Snapshot

```
public interface TransactionCommand extends Serializable
{
    void execute() throws AccountException;
}

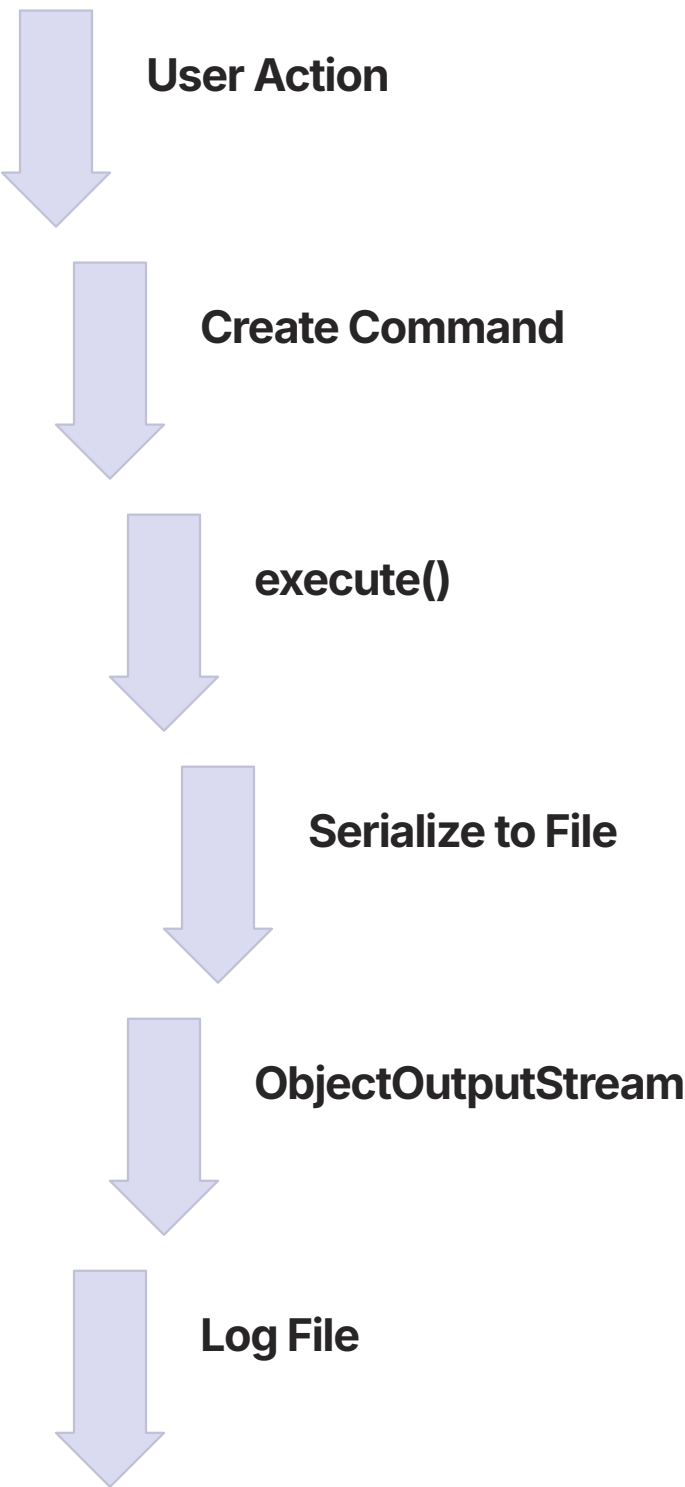
public class DepositCommand implements TransactionCommand {
    private IAccount account;
    private double amount;

    public DepositCommand(IAccount account, double amount) {
        this.account = account;
        this.amount = amount;
    }

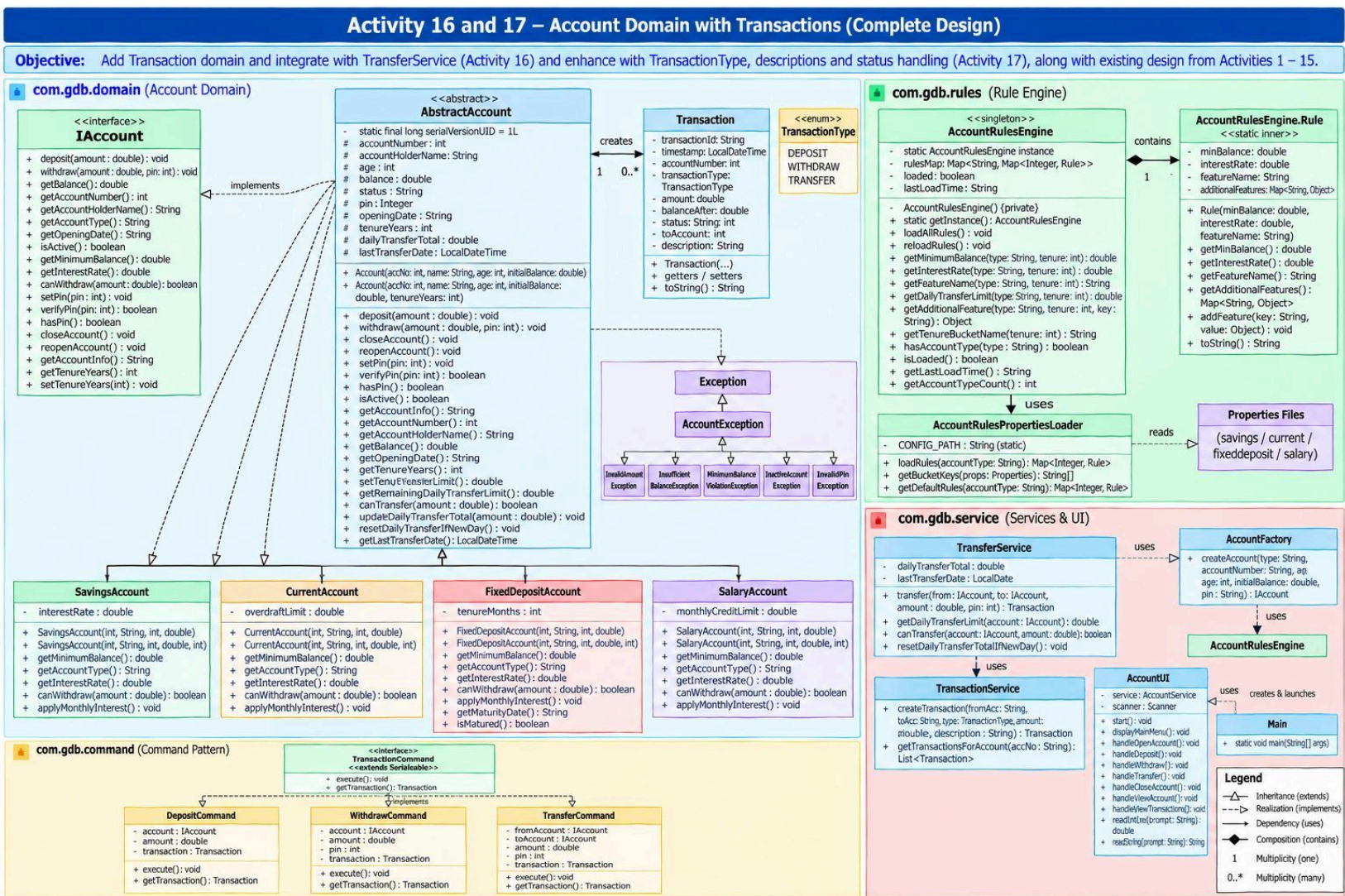
    @Override
    public void execute() throws AccountException {
        account.deposit(amount);
    }
}

public class TransactionLog {
    public void save(TransactionCommand cmd, String filename) throws IOException {
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(filename, true))) {
            oos.writeObject(cmd);
        }
    }
}
```

Below, a stepsWithArrow flow: User Action → Create Command → execute() → Serialize to File → ObjectOutputStream → Log File



Activity 16 & 17 Class Diagram



Bridge Pattern – File + DB

Abstract logging with a LogDestination interface supporting File, Memory, and simulated DB destinations — switchable at runtime.

Key Concepts

- Bridge Pattern
- Abstraction vs Implementation
- Runtime Switching
- Simulated DB

Implementation Elements

- TransactionLogger
- LogDestination (interface)
- FileLogDestination
- DatabaseLogDestination
- MemoryLogDestination
- SimulatedDatabase

Code Snapshot

```
public interface LogDestination {
    void write(Transaction transaction);
    List<Transaction> readAll();
}

public class TransactionLogger {
    private LogDestination destination;

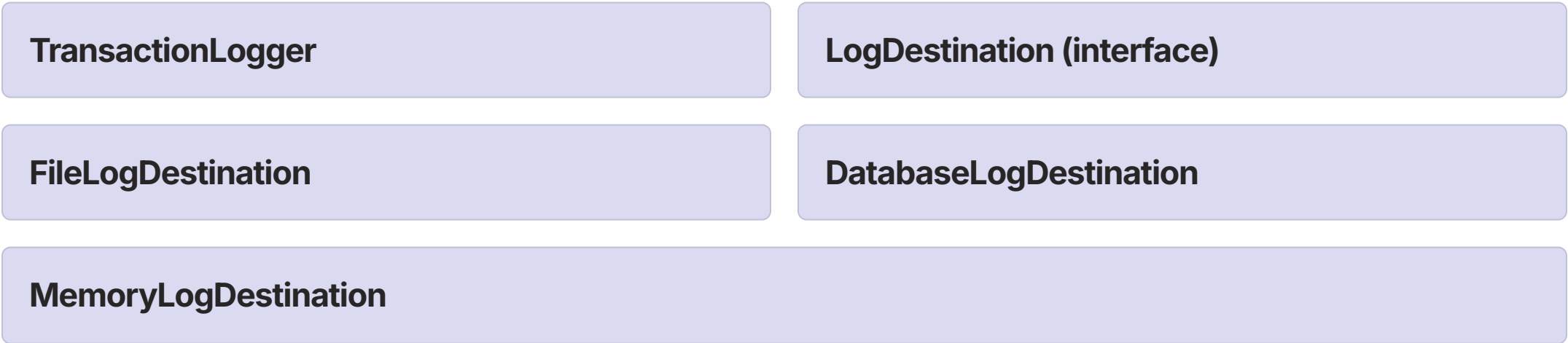
    public TransactionLogger(LogDestination destination) {
        this.destination = destination;
    }

    public void log(Transaction transaction) {
        destination.write(transaction);
    }

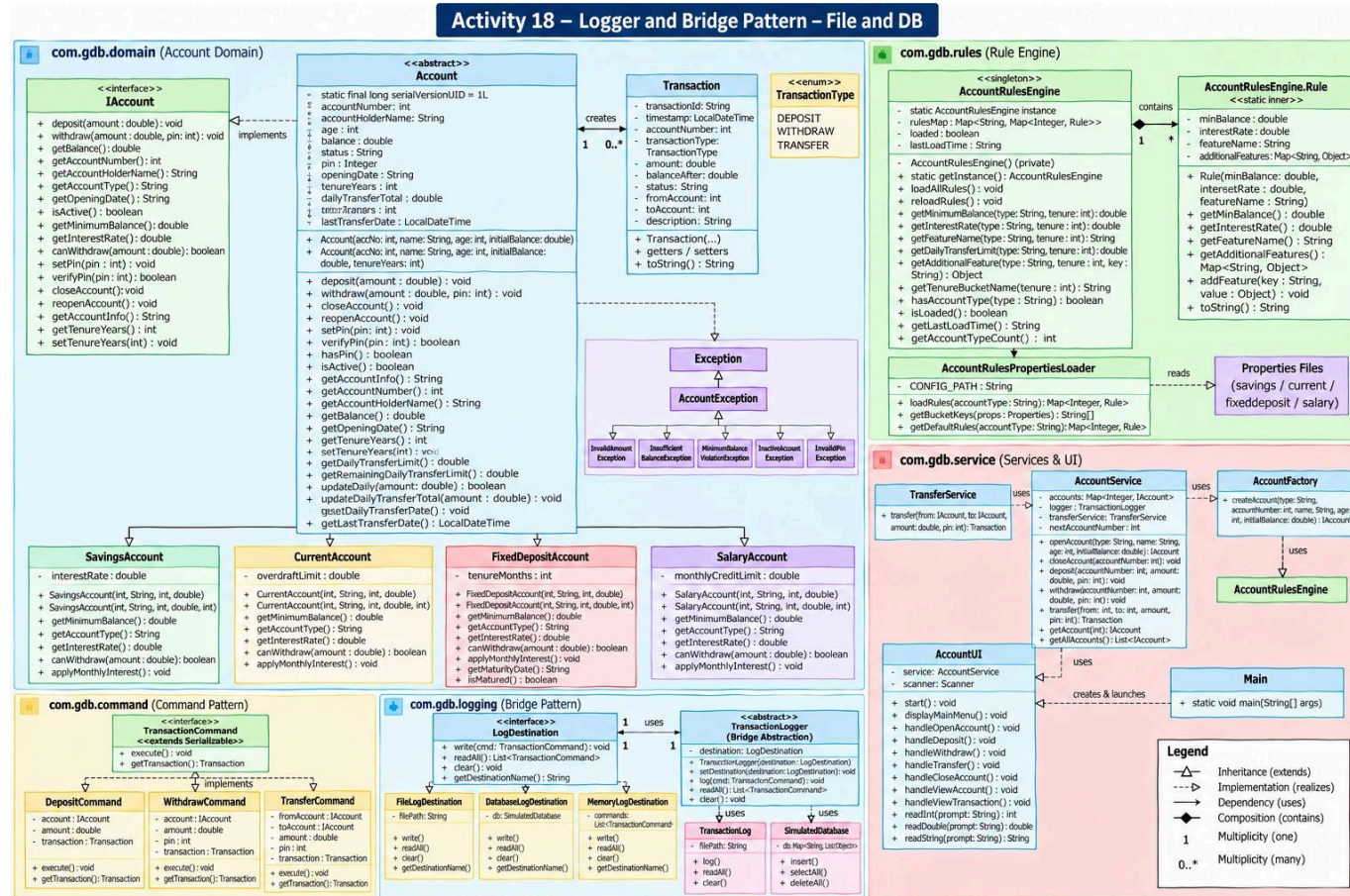
    public void switchDestination(LogDestination
newDestination) {
        this.destination = newDestination;
    }
}

// Usage:
TransactionLogger logger = new TransactionLogger(new
FileLogDestination("log.txt"));
logger.log(transaction);
logger.switchDestination(new
DatabaseLogDestination(simulatedDb));
```

Below, a solidBoxes diagram: TransactionLogger → LogDestination (interface) → FileLogDestination | DatabaseLogDestination | MemoryLogDestination



Activity 18 Class Diagram



AccountService

Introduce AccountService as the single entry-point service layer that orchestrates domain operations and integrates with TransactionLogger.

Key Concepts

- Service Layer
- Orchestration
- Separation of Concerns
- Single Entry Point

Implementation Elements

- AccountService
- account registry (Map<String, IAccount>)
- openAccount()
- closeAccount()
- deposit()
- withdraw()
- transfer()
- integration with TransactionLogger

Code Snapshot

```
public class AccountService {
    private Map<String, IAccount> registry = new
    HashMap<>();
    private TransactionLogger logger;

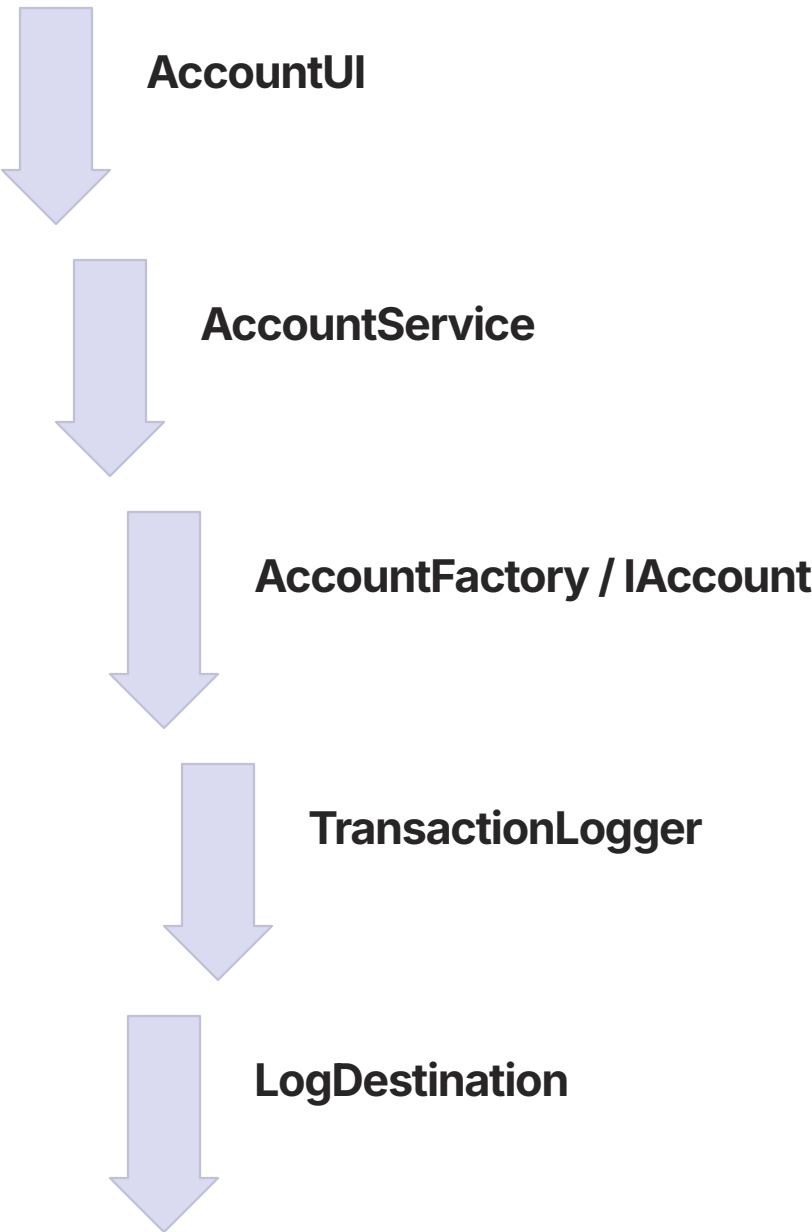
    public AccountService(TransactionLogger logger) {
        this.logger = logger;
    }

    public IAccount openAccount(String type, String accNo,
    String name, double balance) {
        IAccount account =
        AccountFactory.createAccount(type, accNo, name,
        balance);
        registry.put(accNo, account);
        return account;
    }

    public Transaction deposit(String accNo, double
    amount) throws AccountException {
        IAccount account = registry.get(accNo);
        Transaction tx =
        account.depositWithTransaction(amount);
        logger.log(tx);
        return tx;
    }

    public void closeAccount(String accNo) {
        registry.remove(accNo);
    }
}
```

Below, a stepsWithArrow flow: AccountUI → AccountService → AccountFactory / IAccount → TransactionLogger → LogDestination



AccountUI

Build a menu-driven console UI that collects user input and delegates all operations to AccountService, completing the end-to-end system.

Key Concepts

- UI Layer
- Menu Design
- User Input
- Delegation
- End-to-End Flow

Implementation Elements

- AccountUI
- start()
- displayMainMenu()
- Scanner-based input
- handlers for: Open Account, Deposit, Withdraw, Transfer, View Balance, Close Account, Exit.
- Main class launches UI.

Code Snapshot

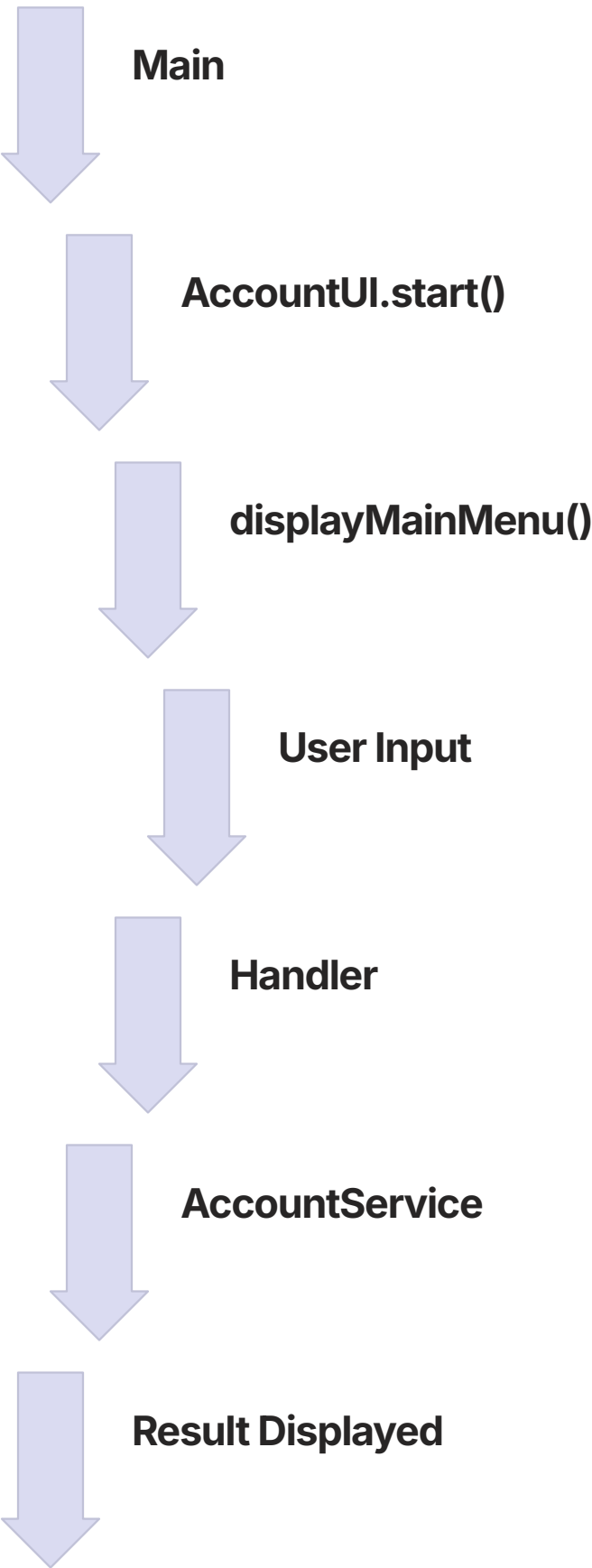
```
public class AccountUI {
    private AccountService service;
    private Scanner scanner = new Scanner(System.in);

    public AccountUI(AccountService service) {
        this.service = service;
    }

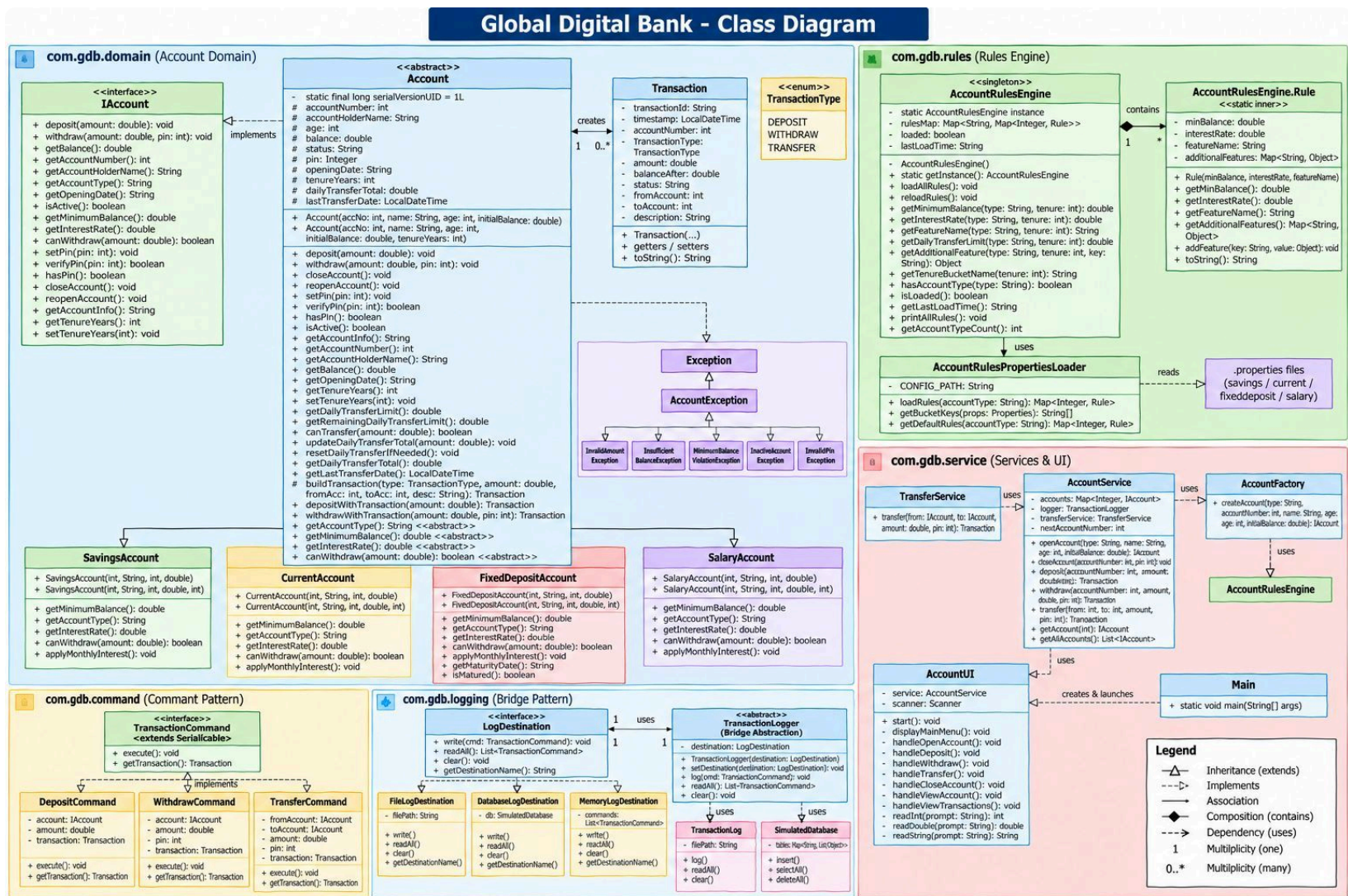
    public void start() {
        while (true) {
            displayMainMenu();
            int choice = Integer.parseInt(scanner.nextLine());
            switch (choice) {
                case 1 -> handleOpenAccount();
                case 2 -> handleDeposit();
                case 3 -> handleWithdraw();
                case 4 -> handleTransfer();
                case 5 -> handleViewBalance();
                case 6 -> handleCloseAccount();
                case 0 -> { System.out.println("Goodbye!"); return; }
            }
        }
    }
}

public class Main {
    public static void main(String[] args) {
        TransactionLogger logger = new TransactionLogger(new FileLogDestination("log.txt"));
        AccountService service = new AccountService(logger);
        new AccountUI(service).start();
    }
}
```

Below, a stepsWithArrow flow: Main → AccountUI.start() → displayMainMenu() → User Input → Handler → AccountService → Result Displayed



Activity 20 Complete Class Diagram



From Zero to Full Stack

20 Activities. One Complete Banking System.

38

Types

9

Packages

3

Interfaces

1

Abstract Class

1

Enums

8

Exceptions

6

Patterns

Students who complete this project will have hands-on experience with core Java OOP principles, design patterns, layered architecture, and real-world banking domain modeling — ready for enterprise Java development.